LEVROMART · ENGINEERING HANDOVER

Backend Build Manual

Everything a backend engineer needs to take over, understand and migrate the Levromart platform: the Supabase database, security model, business logic, scheduled jobs, storage, serverless API layer and operations.

Prepared 07 October 2026 · Live Supabase project `ovklnoroyxobijgvsyfp` · PostgreSQL 17.6
Companion files in the repository: `docs/BUILD_MANUAL.md` (frontend) · `supabase/` (schema docs)

Contents

1. System overview & architecture
2. Environments, access & configuration checklist
3. Data model (71 tables, grouped A–J)
4. Enumerations
5. Security model (RLS, definer functions, advisor findings)
6. Business logic: order, payment, commission, points, messaging, notification flows
7. Scheduled jobs (pg_cron)
8. Functions, triggers & policies catalogue
9. Storage buckets & edge functions
10. Vercel API layer
11. Audit: what was tidied, what still needs a decision
12. Migration playbook (leaving Supabase)
13. Operations runbook (backup, restore, changes)

Appendix: indexes

71 tables (all with RLS)	**311** functions (265 SECURITY DEFINER)	**69** triggers	**149** RLS policies	**186** indexes	**8** pg_cron jobs
11 storage buckets	**13** enum types	**9** extensions	**1** edge function	**12** Vercel API files	**33 MB** database size today

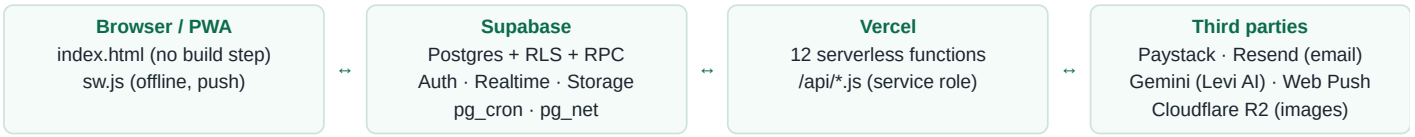
How to read this manual

Sections 1–2 orient you. Section 3 is the data dictionary (every table and column). Sections 5–8 explain behaviour: who may do what, which function enforces which business rule, what runs automatically. Section 11 lists housekeeping that was done and the few items that are intentionally left for you to approve. Section 12 is a practical plan for moving off Supabase. Section 13 is the day-to-day runbook.

> **Golden rule of this backend:** the browser never holds a secret. It uses the public anon key and calls Postgres through Row Level Security and `SECURITY DEFINER` RPC functions that check `auth.uid()` themselves. Anything that needs real secrets (Paystack, email, push, AI, R2) runs in the Vercel functions with the service-role key.

1. System overview & architecture

Levromart is a Nigerian marketplace PWA: anyone can buy; verified vendors can also sell products, services and packages. Customers browse, follow vendors, message and call them, order, and pay the vendor directly (bank transfer reported by the buyer and confirmed by the vendor). Levromart earns commission on completed orders or monthly listing fees. Admins verify vendors, resolve disputes and manage content.



Request paths

- **Normal reads/writes:** browser → Supabase REST (PostgREST) with the user's JWT → RLS decides what rows are visible/changeable.
- **Business actions:** browser → `rpc('function_name')`. Almost all state changes (placing orders, confirming payments, approving vendors, awarding points) are SECURITY DEFINER SQL functions, so rules live next to the data.
- **Secrets-requiring actions:** browser → `/api/*` on Vercel with the user's access token → function verifies the token, then uses the service-role key (Paystack, R2 uploads, Levi AI, payouts, email/push).
- **Database → outside world:** triggers and cron functions call `net.http_post()` (pg_net) to `/api/notify` using a shared webhook secret (`x-webhook-secret`). This is how every notification becomes an email/push.
- **Realtime:** notifications, messages and call signalling use Supabase Realtime (postgres changes + broadcast channels `voicecall-<id>`).

Roles

Role	Where stored	Capabilities
`buyer`	users.role	Browse, cart, order, follow, chat, review, request items. Vendors are also buyers.
`vendor`	users.role + a vendors row owned by the user	List products/services/packages, manage orders, confirm payments, see own analytics. Checked with `owns_vendor(vendor_id)`.
`admin`	users.role = admin; `users.is_super_admin` for super-admins	Verify vendors, handle disputes/support, manage content. Sub-admins have a reduced menu; super-admin-only actions check `is_super_admin()`. Admin accounts cannot buy.

Tech summary

Layer	Technology
Frontend	Single-file vanilla JS PWA (`index.html`), hand-written router, service worker, Web Push
API	Node serverless functions on Vercel Hobby (limit 12 functions — related actions merged behind an `action` field)
Database	Supabase Postgres 17.6; extensions: plpgsql, uuid-ossp, pgcrypto, pg_trgm, btree_gist, pg_net, pg_cron, supabase_vault, pg_stat_statements
Files	Public images on Cloudflare R2; private files in Supabase Storage (signed URLs)
Payments	Mostly direct bank transfer (reported → confirmed). Paystack for listing fees / commission settlement / legacy card orders
Email / push / AI	Resend / web-push (VAPID) / Google Gemini (Levi)

2. Environments, access & configuration checklist

Projects

Name	Project ref	Notes
Live (production)	`ovklnoroyxobijgvsyfp`	The only database the app talks to. Every change described here is applied to this one.
Standby copy	`rtmnrdzmcnlhqfptgkkn`	Created as a spare. **Not kept in sync** — treat as stale. Rebuild it from `supabase/export.sh` output before relying on it.

What you need on day one

- Supabase dashboard access (owner/admin) — Project Settings → Database for the connection string (use the **session pooler, port 5432**) and API keys.
- Vercel project access — environment variables, deployments, logs.
- GitHub repo access — secrets `SUPABASE_DB_URL`, `BACKUP_PASSPHRASE` for the weekly backup workflow; `CRON_SECRET` if the reminder workflow is used.
- Paystack, Resend, Cloudflare (R2), Google AI Studio (Gemini) dashboards.

Secrets and where they live (values are intentionally not in this document)

Secret / key	Lives in	Used by
Supabase URL + anon key	index.html (public by design)	Browser
`SUPABASE_SERVICE_ROLE_KEY`	Vercel env	All /api functions — bypasses RLS
`EMAIL_WEBHOOK_SECRET`	Vercel env **and** hard-coded inside DB functions (`notify_email_on_notification`, `ring_callee_again`, `send_background_reminders`)	Authenticates DB → /api/notify calls. **Must match** in both places; rotating it means redefining those three functions.
`PAYSTACK_SECRET_KEY`	Vercel env	initiate/verify payment, webhook, subaccounts, payouts
`RESEND_API_KEY`, `EMAIL_FROM`	Vercel env (and Supabase secret for the edge function)	Email
`VAPID_PRIVATE_KEY` (+ public key in index.html)	Vercel env	Web push
`GEMINI_API_KEY`	Vercel env	Levi AI + speech transcription
`R2_*` (account id, key id, secret, bucket, public URL)	Vercel env	Public image uploads
`CRON_SECRET`	Vercel env + GitHub secret	Protects /api/daily-cron and /api/order-reminders

> **Security note:** the webhook secret is embedded in function source. Anyone who can read function definitions (database admins) can read it. When you migrate, move it into a secrets manager (see section 12).

Auth

Supabase Auth (schema `auth`) stores credentials and sessions; `public.users` mirrors each account (same `id`) and is created/updated by triggers and by `/api/register`. Admin sign-in adds an emailed one-time code (`admin_login_otps`, verified by `/api/auth-otp`). Manual password resets (`password_reset_requests`) are approved by an admin. Dashboard settings to verify: email provider, Google OAuth, redirect URLs, and **Leaked Password Protection (currently OFF — recommended ON)**.

3. Data model

All 71 application tables live in schema `public`. Every table has Row Level Security enabled. Legend: PK primary key · → table foreign key (with delete rule where not the default) · *not null* · grey = default value. Table comments with the same descriptions are stored in the database itself (visible in the Supabase Table Editor).

Core relationships at a glance

- `auth.users` 1–1 `users` 1–0..1 `vendors` (a vendor row belongs to one user).
- `sectors` 1–n `categories` (parent + sub-category) 1–n `products` n–1 `vendors`. Products 1–n `product_variants`. `vendor_packages` n–n `products` through `package_items`.
- `orders` (buyer, vendor) 1–n `order_items` (→ products/variants), 1–n `payments`, 0..1 `disputes`, 0..1 `reviews`, 0..1 `service_offers`.
- `conversations` (buyer, vendor) 1–n `messages`; `voice_calls` reference a conversation; `notifications` reference a user (and optionally an actor / vendor).
- `vendors` 1–n `commission_charges`, `commission_settlements`, `vendor_monthly_charges`, `monthly_listing_submissions`.
- `points_rules` → `points_ledger` → `points_weekly` → `vendor_week_winners`.

A — Accounts, identity & security

Who the users are, how they sign in, and the audit trail around them. Supabase Auth owns the passwords and sessions (schema auth); public.users is the application profile that mirrors it (same id).

`users`

Application profile for every account (buyer, vendor, admin). Same id as auth.users. Holds role, contact info, suspension state, legal-terms acceptance, super-admin flag.

id	uuid *not null*	PK → auth.users cascade	
name	text *not null*		''::text
phone	text		
email	text		
role	user_role *not null*		'buyer'::user_role
city	text		
created_at	timestamp with time zone *not null*		now()
avatar_url	text		
address	text		
ai_data_consent	boolean *not null*		false
role_selected	boolean *not null*		true
suspended_at	timestamp with time zone		
suspended_reason	text		
suspended_by	uuid	→ users	
is_super_admin	boolean *not null*		false
has_password	boolean *not null*		false
otp_verified_at	timestamp with time zone *not null*		now()
legal_terms_version	text		
legal_terms_accepted_at	timestamp with time zone		

RLS **on** · 2 policies · triggers: trg_notify_new_user_signup, trg_prevent_self_role_change, trg_sync_vendor_avatar_from_user, trg_sync_vendor_profile_avatar, trg_user_profile_updated

`blocked_users`

Pairs (blocker, blocked) that stop two users messaging each other.

blocker_id	uuid *not null*	PK → auth.users cascade	
blocked_id	uuid *not null*	PK → auth.users cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies

`account_appeals`

Appeals written by suspended users; reviewed by admins.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
suspension_reason	text		
message	text *not null*		
status	text *not null*		'pending'::text
admin_response	text		
created_at	timestamp with time zone *not null*		now()
resolved_at	timestamp with time zone		
resolved_by	uuid	→ users	

RLS **on** · 2 policies

admin_actions

Audit log of admin actions (who did what to which record).

id	uuid *not null*	PK	gen_random_uuid()
admin_id	uuid *not null*	→ users	
admin_name	text		
action	text *not null*		
target_type	text *not null*		
target_id	uuid		
target_label	text		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 2 policies · triggers: trg_notify_admins_on_action

admin_login_otps

One-time email codes for admin login (verified by api/auth-otp.js). Short-lived, no client access.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users	
code	text *not null*		
expires_at	timestamp with time zone *not null*		
used_at	timestamp with time zone		
created_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

password_reset_requests

Manual password-reset requests approved by admins when email reset is not possible. Token is stored hashed.

id	uuid *not null*	PK	gen_random_uuid()
email	text *not null*		
user_id	uuid	→ auth.users set null	
status	text *not null*		'pending'::text
token_hash	text *not null*		
created_at	timestamp with time zone *not null*		now()
approved_at	timestamp with time zone		
approved_by	uuid	→ auth.users set null	
rejected_at	timestamp with time zone		
rejected_by	uuid	→ auth.users set null	
completed_at	timestamp with time zone		
expires_at	timestamp with time zone *not null*		(now() + '00:30:00'::interval)

RLS **on** · no policies (server/RPC access only)

auth_activity_logs

Sign-in / sign-out / session events for the admin 'Login logs' page.

id	uuid *not null*	PK	gen_random_uuid()
session_id	uuid		
user_id	uuid *not null*	→ auth.users cascade	
event_type	text *not null*		
occurred_at	timestamp with time zone *not null*		now()
user_agent	text		
ip	inet		
source	text *not null*		'app'::text
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

push_subscriptions

Web-push endpoints per user and device (endpoint, p256dh, auth_key). Used by api/notify.js.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ auth.users cascade	
endpoint	text *not null*		
p256dh	text *not null*		
auth_key	text *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies

`user_presence`

Last-seen timestamp per user (touch_last_seen RPC, every ~45 s). Drives 'last seen' in chats.

user_id	uuid *not null*	PK → users cascade	
last_seen_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

`app_presence`

'Is the app open right now' heartbeat (touch_presence / clear_presence RPCs). api/notify.js skips system pushes for 45 s after the last beat so notifications stay in-app. Different purpose from user_presence.

user_id	uuid *not null*	PK	
seen_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

B — Vendors & storefronts

A vendor is a user with a row in vendors. Verification, billing model, payout details, delivery settings and storefront data all live on that row.

`vendors`

One row per shop: business details, verification status, bank/Paystack codes, commission and monthly-billing state, delivery settings, location, ratings, soft-delete fields.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
business_name	text *not null*		
logo_url	text		
cover_image_url	text		
bio	text		
address	text		
city	text		
phone	text		
whatsapp_number	text		
verification_status	vendor_verification_status *not null*		'pending'::vendor_verification_status
cac_number	text		
verification_docs	_text[] *not null*		'{}'::text[]
delivery_radius_km	numeric		
opening_hours	text		
avg_rating	numeric		
created_at	timestamp with time zone *not null*		now()
is_active	boolean *not null*		true
bank_code	text		
bank_name	text		
account_number	text		
account_name	text		
paystack_subaccount_code	text		
commission_pct	numeric *not null*		3.00
delivery_fee	numeric *not null*		0
sector_id	uuid *not null*	→ sectors	
deleted_at	timestamp with time zone		
transfer_recipient_code	text		
billing_model	text *not null*		'commission'::text
paystack_authorization_code	text		
paystack_customer_code	text		
monthly_billing_status	text *not null*		'inactive'::text
next_monthly_charge_at	timestamp with time zone		
review_count	integer *not null*		0
lat	double precision		
lng	double precision		
business_hours	jsonb		
allow_after_hours_messages	boolean *not null*		true
delivery_fee_mode	text *not null*		'flat'::text
delivery_fee_per_km	numeric *not null*		0
previous_business_name	text		
name_changed_at	timestamp with time zone		
whatsapp_visible	boolean *not null*		false
cover_url	text		
commission_paused	boolean *not null*		false
commission_pause_reason	text		
deleted_previous_verification_status	vendor_verification_status		
deleted_previous_is_active	boolean		
admin_paused	boolean *not null*		false
profile_view_count	integer *not null*		0
referral_code	text		

RLS **on** · 3 policies · triggers: trg_enforce_vendor_onboarding_requirements, trg_notify_new_vendor_application, trg_notify_vendor_payout_setup, trg_points_vendor, trg_prevent_vendor_activation_with_owed_commission, trg_sync_user_role_from_vendor, trg_validate_vendor_phone_numbers, trg_vendor_profile_updated, trg_vendor_referral_code

`sectors`

Top-level business sectors (e.g. Fashion & Beauty). Vendors can create custom sectors.

id	uuid *not null*	PK	gen_random_uuid()
slug	text *not null*		
name	text *not null*		
icon	text *not null*		'fa-shapes'::text
sort_order	integer *not null*		0

RLS **on** · 1 policy

`categories`

Two-level tree under a sector: parent_category_id is null for a parent ('Other'), set for sub-categories. Products point at sub-categories. A trigger gives uncategorised products a default sub-category named after the vendor's sector.

id	uuid *not null*	PK	gen_random_uuid()
name	text *not null*		
parent_category_id	uuid	→ categories set null	
sector_id	uuid *not null*	→ sectors	

RLS **on** · 5 policies

`vendor_follows`

Customer follows vendor.

id	uuid *not null*	PK	gen_random_uuid()
follower_id	uuid *not null*	→ users cascade	
vendor_id	uuid *not null*	→ vendors cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies · triggers: trg_points_follow, vendor_follow_notify_vendor_trigger

`vendor_follow_preferences`

Per-follow notification preferences (new listing, price change, discount, mute window).

id	uuid *not null*	PK	gen_random_uuid()
follower_id	uuid *not null*	→ users cascade	
vendor_id	uuid *not null*	→ vendors cascade	
muted_until	timestamp with time zone		
mute_mode	text *not null*		'none'::text
notify_new_listing	boolean *not null*		true
notify_new_service	boolean *not null*		true
notify_new_package	boolean *not null*		true
notify_new_variety	boolean *not null*		true
notify_price_change	boolean *not null*		true
notify_discount	boolean *not null*		true
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies

`vendor_profile_view_events`

One row per storefront view; powers 'Views today' and analytics.

id	bigint *not null*	PK	
vendor_id	uuid *not null*	→ vendors cascade	
viewed_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`vendor_recycle_bin`

Snapshot taken when an admin trashes a vendor so it can be restored exactly.

vendor_id	uuid *not null*	PK → vendors cascade	
deleted_at	timestamp with time zone *not null*		now()
deleted_by	uuid	→ users set null	
product_statuses	jsonb *not null*		'[]'::jsonb
package_statuses	jsonb *not null*		'[]'::jsonb
created_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

C — Catalogue, engagement & search

Products, services and packages, plus everything customers do around them.

`products`

Sellable items. is_service marks services (commission/negotiated); listing_type = sale / rent / both; billing_* columns track monthly-listing payment; payment_timing = pay_before / pay_after with payment_deposit_percent.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
category_id	uuid	→ categories set null	
name	text *not null*		
description	text		
photos	_text[] *not null*		'{}'::text[]
condition	product_condition *not null*		'used-good'::product_condition
listing_type	listing_type *not null*		
sale_price	numeric		
rental_rate	numeric		
rental_rate_unit	text		
deposit_amount	numeric		
quantity_total	integer *not null*		1
quantity_available	integer *not null*		1
status	product_status *not null*		'active'::product_status
created_at	timestamp with time zone *not null*		now()
view_count	integer *not null*		0
is_service	boolean *not null*		false
service_price_unit	text		
discount_price	numeric		
delivery_fee_mode	text *not null*		'free'::text
delivery_fee	numeric *not null*		0
delivery_fee_per_km	numeric *not null*		0
billing_model	text *not null*		'commission'::text
billing_status	text *not null*		'active'::text
billing_paid_until	timestamp with time zone		
billing_reference	text		
payment_timing	text *not null*		'pay_before'::text
payment_deposit_percent	numeric *not null*		20
admin_deleted_at	timestamp with time zone		
admin_deleted_by	uuid	→ users	
track_quantity	boolean *not null*		false

RLS **on** · 4 policies · triggers: trg_notify_low_stock_products, trg_points_listing, trg_products_default_category, trg_products_notify_followers, trg_products_remove_monthly_submission

`product_variants`

Varieties of a product (own name, photo, price, size, colour, stock).

id	uuid *not null*	PK	gen_random_uuid()
product_id	uuid *not null*	→ products cascade	
name	text *not null*		
photo_url	text		
sale_price	numeric		
discount_price	numeric		
rental_rate	numeric		
sort_order	integer *not null*		0
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()
quantity_available	integer *not null*		0
size	text		
color	text		
description	text		
quantity_total	integer		

RLS **on** · 4 policies · triggers: trg_notify_low_stock_variants, trg_product_variants_notify_followers

`vendor_packages`

Bundles of products sold at one price.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
name	text *not null*		
description	text		
price	numeric *not null*		
status	product_status *not null*		'active'::product_status
created_at	timestamp with time zone *not null*		now()
photos	jsonb *not null*		'[]'::jsonb
billing_model	text *not null*		'commission'::text
billing_status	text *not null*		'active'::text
billing_paid_until	timestamp with time zone		
billing_reference	text		

RLS **on** · 4 policies · triggers: trg_packages_remove_monthly_submission, trg_points_listing, trg_vendor_packages_notify_followers

`package_items`

Which products (and quantities) make up a package.

id	uuid *not null*	PK	gen_random_uuid()
package_id	uuid *not null*	→ vendor_packages cascade	
product_id	uuid *not null*	→ products cascade	
quantity	integer *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies

`favorites`

Customer wishlist (products).

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ users cascade	
product_id	uuid *not null*	→ products cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies · triggers: favorites_notify_product_wishlist

`package_favorites`

Customer wishlist (packages).

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ auth.users cascade	
package_id	uuid *not null*	→ vendor_packages cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies

`product_comments`

Comments and replies on products.

id	uuid *not null*	PK	gen_random_uuid()
product_id	uuid *not null*	→ products cascade	
user_id	uuid *not null*	→ users cascade	
parent_comment_id	uuid	→ product_comments cascade	
content	text *not null*		
author_name	text *not null*		
author_avatar_url	text		
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: product_comments_notify_event, product_comments_snapshot_author

`product_comment_likes`

Likes on product comments.

id	uuid *not null*	PK	gen_random_uuid()
comment_id	uuid *not null*	→ product_comments cascade	
user_id	uuid *not null*	→ users cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies · triggers: product_comment_likes_notify_event

`product_engagement_settings`

Vendor controls per product: allow comments, who may comment, like visibility.

product_id	uuid *not null*	PK → products cascade	
vendor_id	uuid *not null*	→ vendors cascade	
like_visibility	text *not null*		'everyone'::text
allow_comments	boolean *not null*		true
comment_audience	text *not null*		'everyone'::text
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: product_engagement_settings_updated_at

`package_comments`

Comments and replies on packages.

id	uuid *not null*	PK	gen_random_uuid()
package_id	uuid *not null*	→ vendor_packages cascade	
user_id	uuid *not null*	→ users cascade	
parent_comment_id	uuid	→ package_comments cascade	
content	text *not null*		
author_name	text *not null*		'Levromart user'::text
author_avatar_url	text		
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: package_comments_updated_at

`package_comment_likes`

Likes on package comments.

id	uuid *not null*	PK	gen_random_uuid()
comment_id	uuid *not null*	→ package_comments cascade	
user_id	uuid *not null*	→ users cascade	
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies

`package_engagement_settings`

Vendor controls per package (same options as products).

package_id	uuid *not null*	PK → vendor_packages cascade	
vendor_id	uuid *not null*	→ vendors cascade	
like_visibility	text *not null*		'everyone'::text
allow_comments	boolean *not null*		true
comment_audience	text *not null*		'everyone'::text
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: package_engagement_settings_updated_at

`reviews`

Star rating + comment left by a buyer after a completed order. Triggers keep vendors.avg_rating / review_count current.

id	uuid *not null*	PK	gen_random_uuid()
order_id	uuid *not null*	→ orders cascade	
buyer_id	uuid *not null*	→ users	
vendor_id	uuid *not null*	→ vendors	
rating	integer *not null*		
comment	text		
created_at	timestamp with time zone *not null*		now()
buyer_name	text		
buyer_avatar_url	text		

RLS **on** · 3 policies · triggers: trg_notify_new_review, trg_points_review, trg_set_review_buyer_snapshot, trg_sync_vendor_review_stats

`search_synonyms`

Synonym pairs used by search_products_full_v2 / search_product_ids.

term	text *not null*	PK
synonym	text *not null*	PK

RLS **on** · 1 policy

`bookings`

Date ranges reserved for rental order items (exclusion constraint prevents double booking).

id	uuid *not null*	PK	gen_random_uuid()
product_id	uuid *not null*	→ products cascade	
order_item_id	uuid *not null*	→ order_items cascade	
date_range	daterange *not null*		
quantity	integer *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`inventory`

Legacy healthcare-equipment / blood-stock listings from the first version of the app. Not used by the marketplace flows; candidate for removal.

id	uuid *not null*	PK	gen_random_uuid()
type	text *not null*		
category	text *not null*		
item_name	text *not null*		
blood_group	text		
quantity	integer *not null*		1
urgency	text *not null*		
hospital_name	text *not null*		
phone	text *not null*		
lat	double precision		
lng	double precision		
description	text		
verified	boolean *not null*		false
created_at	timestamp with time zone *not null*		now()
owner_id	uuid	→ auth.users	

RLS **on** · 4 policies

D — Cart, orders & disputes

Checkout and the order lifecycle. Status flow: pending > confirmed > (handed_over) > completed, with cancelled, disputed and payment_unresolved side-branches. Payments are mostly direct bank transfers reported by the buyer and confirmed by the vendor.

`cart_items`

Items a buyer has in the cart (product + optional variant + quantity).

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ users cascade	
product_id	uuid *not null*	→ products cascade	
quantity	integer *not null*		1
created_at	timestamp with time zone *not null*		now()
variant_id	uuid	→ product_variants cascade	

RLS **on** · 1 policy · triggers: trg_prevent_vendor_self_cart_item

orders

One order per vendor. Money fields, delivery fields, payment_timing / payment_deposit_percent, status timestamps, payout fields (legacy card flow), awaiting_confirmation_since (24 h auto-complete timer).

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ users	
vendor_id	uuid *not null*	→ vendors	
order_type	order_type *not null*		
status	order_status *not null*		'pending'::order_status
total_amount	numeric *not null*		0
deposit_amount	numeric *not null*		0
delivery_method	delivery_method *not null*		
delivery_address	text		
created_at	timestamp with time zone *not null*		now()
delivery_phone	text		
delivery_fee	numeric *not null*		0
package_id	uuid	→ vendor_packages	
payout_status	text *not null*		'not_applicable'::text
auto_release_at	timestamp with time zone		
payout_released_at	timestamp with time zone		
payout_transfer_code	text		
awaiting_confirmation_since	timestamp with time zone		
last_reminder_sent_at	timestamp with time zone		
buyer_note	text		
status_changed_at	timestamp with time zone		
completed_at	timestamp with time zone		
cancelled_at	timestamp with time zone		
auto_completed_at	timestamp with time zone		
payment_timing	text *not null*		'pay_before'::text
payment_deposit_percent	numeric *not null*		20
buyer_lat	double precision		
buyer_lng	double precision		
service_offer_id	uuid	→ service_offers set null	
commission_exempt	boolean *not null*		false
payment_unresolved_at	timestamp with time zone		
payment_escalated_at	timestamp with time zone		
cancellation_reason	text		
quantity_proposal	jsonb		

RLS **on** · 3 policies · triggers: trg_guard_unverified_order_completion, trg_levromart_stamp_order_status_clock, trg_normalize_order_payment_terms, trg_notify_new_order, trg_notify_order_status_changed, trg_orders_require_buyer_phone, trg_points_order, trg_prevent_unverified_order_completion, trg_prevent_vendor_self_order, trg_record_order_completion_commission, trg_require_buyer_phone_for_order, trg_restore_product_stock_on_cancelled_sale

order_items

Line items of an order with the unit price at purchase time.

id	uuid *not null*	PK	gen_random_uuid()
order_id	uuid *not null*	→ orders cascade	
product_id	uuid *not null*	→ products	
quantity	integer *not null*		
unit_price	numeric *not null*		
rental_start_date	date		
rental_end_date	date		
created_at	timestamp with time zone *not null*		now()
variant_id	uuid	→ product_variants set null	

RLS **on** · 2 policies · triggers: trg_sync_order_payment_deposit_percent

order_notes

Free-text notes exchanged on an order.

id	uuid *not null*	PK	gen_random_uuid()
order_id	uuid *not null*	→ orders cascade	
sender_id	uuid *not null*	→ users cascade	
body	text *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

payments

Payment records per order: sale_price, rental_fee, deposit, delivery_deposit_20 (the pay-after deposit). status pending > paid. provider_reference tells how it was confirmed (buyer_report, vendor_confirmed, admin_confirmed, ...).

id	uuid *not null*	PK	gen_random_uuid()
order_id	uuid *not null*	→ orders cascade	
payment_type	payment_type *not null*		
amount	numeric *not null*		
method	payment_method *not null*		
status	payment_status *not null*		'pending'::payment_status
provider_reference	text		
created_at	timestamp with time zone *not null*		now()
paid_at	timestamp with time zone		

RLS **on** · 1 policy · triggers: trg_mark_order_payment_unresolved_from_claim, trg_notify_payment_paid

disputes

Issues raised on an order by a buyer or vendor; admin resolves.

id	uuid *not null*	PK	gen_random_uuid()
order_id	uuid *not null*	→ orders cascade	
raised_by	dispute_raised_by *not null*		
reason	text *not null*		
status	dispute_status *not null*		'open'::dispute_status
admin_notes	text		
created_at	timestamp with time zone *not null*		now()
vendor_response	text		
attachments	jsonb *not null*		'[]'::jsonb

RLS **on** · 3 policies · triggers: trg_notify_new_dispute

service_offers

Negotiation thread for a service listing (price rounds, accept, payment reported, completed). Creates an order when accepted.

id	uuid *not null*	PK	gen_random_uuid()
listing_id	uuid *not null*	→ products cascade	
buyer_id	uuid *not null*	→ users cascade	
vendor_id	uuid *not null*	→ vendors cascade	
description	text *not null*		
current_price	numeric *not null*		
current_offer_by	text *not null*		
status	text *not null*		'pending'::text
round_count	integer *not null*		1
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()
expires_at	timestamp with time zone *not null*		(now() + '72:00:00'::interval)
accepted_at	timestamp with time zone		
payment_reported_at	timestamp with time zone		
completed_at	timestamp with time zone		
order_id	uuid	→ orders set null	
payment_reminder_sent_at	timestamp with time zone		
vendor_confirmation_reminder_sent_at	timestamp with time zone		

RLS **on** · 1 policy · triggers: trg_touch_service_offer_updated_at

`equipment_requests`

'Request an item' posts from customers (also called customer requests).

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ users cascade	
title	text *not null*		
description	text		
category_id	uuid	→ categories	
city	text		
needed_from	date		
needed_to	date		
status	text *not null*		'open'::text
created_at	timestamp with time zone *not null*		now()
request_type	text *not null*		'rent'::text
buyer_name	text		
buyer_avatar_url	text		

RLS **on** · 3 policies · triggers: trg_notify_new_request, trg_notify_request_cancelled, trg_set_request_buyer_snapshot

`request_offers`

Vendor offers made on a customer request.

id	uuid *not null*	PK	gen_random_uuid()
request_id	uuid *not null*	→ equipment_requests cascade	
vendor_id	uuid *not null*	→ vendors cascade	
message	text		
price	numeric		
status	text *not null*		'pending'::text
created_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies · triggers: trg_notify_new_offer, trg_notify_offer_accepted

E — Commission & vendor billing

How Levromart earns: commission on completed orders for commission vendors, or monthly listing fees for monthly vendors.

`commission_charges`

Commission owed per completed order.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
payment_id	uuid	→ payments set null	
order_id	uuid	→ orders set null	
amount	numeric *not null*		
status	text *not null*		'owed'::text
created_at	timestamp with time zone *not null*		now()
paid_at	timestamp with time zone		
due_at	timestamp with time zone		(now() + '7 days'::interval)

RLS **on** · 1 policy

`commission_settlements`

Payments a vendor makes to settle commission.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
amount	numeric *not null*		
status	text *not null*		'pending'::text
provider_reference	text		
created_at	timestamp with time zone *not null*		now()
paid_at	timestamp with time zone		

RLS **on** · 1 policy

`commission_deadline_reminders`

De-duplication log so each deadline reminder is sent once.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
reminder_code	text *not null*		
period_key	timestamp with time zone *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

`vendor_monthly_charges`

Monthly billing runs per vendor.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors	
listing_count	integer *not null*		
amount	numeric *not null*		
status	text *not null*		'pending'::text
provider_reference	text		
failure_reason	text		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`monthly_listing_submissions`

Receipts a vendor uploads to activate a monthly listing; admin approves or rejects.

id	uuid *not null*	PK	gen_random_uuid()
kind	text *not null*		
listing_id	uuid *not null*		
vendor_id	uuid *not null*	→ vendors cascade	
user_id	uuid *not null*	→ auth.users cascade	
payer_name	text *not null*		
receipt_path	text *not null*		
status	text *not null*		'pending'::text
submitted_at	timestamp with time zone *not null*		now()
reviewed_at	timestamp with time zone		
reviewed_by	uuid	→ auth.users	
review_note	text		
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 3 policies

`listing_payment_intents`

Paystack payment intents for listing activation.

id	uuid *not null*	PK	gen_random_uuid()
reference	text *not null*		
user_id	uuid *not null*	→ auth.users cascade	
vendor_id	uuid *not null*	→ vendors cascade	
kind	text *not null*		
amount_naira	numeric *not null*		999
status	text *not null*		'pending'::text
listing_id	uuid		
created_at	timestamp with time zone *not null*		now()
paid_at	timestamp with time zone		
consumed_at	timestamp with time zone		
draft_data	jsonb		

RLS **on** · no policies (server/RPC access only)

`listing_drafts`

Unfinished listings saved while a vendor pays.

id	uuid *not null*	PK	gen_random_uuid()
vendor_id	uuid *not null*	→ vendors cascade	
kind	text *not null*		
draft_data	jsonb *not null*		'{}'::jsonb
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: listing_drafts_touch_updated_at

`listing_billing_reminders`

De-duplication log for listing-billing reminders.

kind	text *not null*	PK	
listing_id	uuid *not null*	PK	
reminder_code	text *not null*	PK	
period_key	date *not null*	PK	
sent_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

F — Messaging & calls

In-app chat (one thread per buyer-vendor pair, open only around an order) and WebRTC voice calls (signalling over Supabase Realtime).

`conversations`

A buyer-vendor thread with pin/unread flags per side.

id	uuid *not null*	PK	gen_random_uuid()
buyer_id	uuid *not null*	→ users	
vendor_id	uuid *not null*	→ vendors	
created_at	timestamp with time zone *not null*		now()
pinned_by_buyer	boolean *not null*		false
pinned_by_vendor	boolean *not null*		false
marked_unread_by_buyer	boolean *not null*		false
marked_unread_by_vendor	boolean *not null*		false

RLS **on** · 2 policies

`messages`

Chat messages (text, attachments, reactions, replies, edit/delete flags). Immutable for evidence: edits are limited by trigger.

id	uuid *not null*	PK	gen_random_uuid()
conversation_id	uuid *not null*	→ conversations cascade	
sender_id	uuid *not null*	→ users	
body	text *not null*		
created_at	timestamp with time zone *not null*		now()
read_at	timestamp with time zone		
delivered_at	timestamp with time zone		
reactions	jsonb *not null*		'{}'::jsonb
attachment_url	text		
attachment_name	text		
attachment_type	text		
attachment_size	integer		
edited_at	timestamp with time zone		
deleted_at	timestamp with time zone		
reply_to_id	uuid	→ messages set null	

RLS **on** · 3 policies · triggers: trg_notify_new_message

`message_pins`

Pinned messages with expiry.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ auth.users cascade	
conversation_id	uuid *not null*	→ conversations cascade	
message_id	uuid *not null*	→ messages cascade	
pinned_at	timestamp with time zone *not null*		now()
expires_at	timestamp with time zone		

RLS **on** · 3 policies · triggers: trg_message_pins_limit

`chat_archive`

Private archive of chats removed 24 h after an order completes (kept 30 days).

id	uuid *not null*	PK	gen_random_uuid()
conversation_id	uuid *not null*		
buyer_id	uuid		
vendor_id	uuid		
conversation	jsonb *not null*		
messages	jsonb *not null*		
archived_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

`chat_lists`

User-made chat folders ('lists').

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
name	text *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`chat_list_conversations`

Which conversations belong to which list.

list_id	uuid *not null*	PK → chat_lists cascade	
conversation_id	uuid *not null*	PK → conversations cascade	
added_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`voice_calls`

Call log and state (ringing > active > ended / declined / missed) plus ring counter.

id	uuid *not null*	PK	gen_random_uuid()
conversation_id	uuid *not null*	→ conversations cascade	
caller_id	uuid *not null*	→ users cascade	
callee_id	uuid *not null*	→ users cascade	
status	text *not null*		'ringing'::text
created_at	timestamp with time zone *not null*		now()
answered_at	timestamp with time zone		
ended_at	timestamp with time zone		
rings	integer *not null*		0

RLS **on** · 1 policy

G — Notifications

Every in-app notification is a row; a database trigger posts it to /api/notify which sends email and push.

`notifications`

In-app notifications (type, title, body, target_view + target_params for deep links, read/archived/deleted flags). Largest table; types include new_message, order_status_changed, vendor_follow_update, incoming_call, points, rank_change.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
type	text *not null*		
title	text *not null*		
body	text		
target_view	text		
target_params	jsonb *not null*		'{}'::jsonb
actor_id	uuid	→ users set null	
actor_name	text		
actor_avatar_url	text		
read	boolean *not null*		false
created_at	timestamp with time zone *not null*		now()
archived	boolean *not null*		false
deleted_at	timestamp with time zone		
vendor_id	uuid	→ vendors set null	
notification_category	text *not null*		'system'::text
group_key	text		
metadata	jsonb *not null*		'{}'::jsonb

RLS **on** · 4 policies · triggers: trg_notify_email

H — Points, leaderboard & referrals

Gamification: users earn points for actions; weekly totals decide the leaderboard and the vendor-of-the-week.

`points_rules`

Configurable rules: kind, points, audience, daily/weekly caps.

kind	text *not null*	PK	
label	text *not null*		
description	text *not null*		
points	integer *not null*		
audience	text *not null*		'vendor'::text
category	text *not null*		'growth'::text
icon	text *not null*		'fa-star'::text
daily_cap	integer		
weekly_cap	integer		
sort_order	integer *not null*		100
active	boolean *not null*		true
min_amount	numeric		
per_source_weekly_cap	integer		

RLS **on** · 1 policy

`points_ledger`

Every award (event_key makes awards idempotent).

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
kind	text *not null*		
points	integer *not null*		
event_key	text *not null*		
source_user_id	uuid		
note	text		
meta	jsonb *not null*		'{}'::jsonb
week_start	date *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`points_weekly`

Rolled-up weekly totals (also used for rank-change notifications).

user_id	uuid *not null*	PK → users cascade	
week_start	date *not null*	PK	
points	integer *not null*		0
tx_count	integer *not null*		0
last_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy · triggers: trg_points_rank_notify

`points_skips`

Why an award was skipped (cap reached, duplicate, ...).

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid	→ users cascade	
kind	text *not null*		
ref	text *not null*		
reason	text *not null*		
created_at	timestamp with time zone *not null*		now()

RLS **on** · no policies (server/RPC access only)

`vendor_week_winners`

Snapshot of each finished week's winners.

week_start	date *not null*	PK	
user_id	uuid *not null*	→ users cascade	
vendor_id	uuid *not null*	→ vendors cascade	
points	integer *not null*		
tx_count	integer *not null*		0
top3	jsonb *not null*		'[]'::jsonb
reward_status	text *not null*		'pending'::text
finalized_at	timestamp with time zone *not null*		now()

RLS **on** · 1 policy

`referrals`

Who referred whom (referral codes live on vendors.referral_code).

referred_user_id	uuid *not null*	PK → users cascade	
referrer_user_id	uuid *not null*	→ users cascade	
referrer_vendor_id	uuid *not null*	→ vendors cascade	
code	text *not null*		
created_at	timestamp with time zone *not null*		now()
vendor_awarded	boolean *not null*		false

RLS **on** · 1 policy

I — Levi (AI assistant) & usage limits

Chat history for the Levi assistant (api/chat.js) and per-day usage counters.

`ai_conversations`

Levi conversations per user (optional share token).

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ users cascade	
title	text		
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()
share_token	uuid		

RLS **on** · 4 policies

`ai_messages`

Messages in a Levi conversation, with reaction and attachment fields.

id	uuid *not null*	PK	gen_random_uuid()
conversation_id	uuid *not null*	→ ai_conversations cascade	
role	text *not null*		
content	text *not null*		
created_at	timestamp with time zone *not null*		now()
reaction	text		
attachment_url	text		
attachment_name	text		
attachment_type	text		
attachment_size	integer		

RLS **on** · 3 policies · triggers: trg_touch_ai_conversation

`chat_usage`

Messages sent to Levi per user per day (limit enforced by increment_chat_usage).

user_id	uuid *not null*	PK → users	
day	date *not null*	PK	CURRENT_DATE
count	integer *not null*		0

RLS **on** · no policies (server/RPC access only)

`tts_usage`

Text-to-speech characters used per user per day.

user_id	uuid *not null*	PK → auth.users cascade	
day	date *not null*	PK	CURRENT_DATE
chars	integer *not null*		0

RLS **on** · no policies (server/RPC access only)

J — Site content & support

Content the admins manage and the feedback channels.

`homepage_carousel_slides`

Home-page banner slides (image in storage bucket homepage-carousel).

id	uuid *not null*	PK	gen_random_uuid()
storage_path	text *not null*		
image_url	text *not null*		
link_url	text		
alt_text	text		
sort_order	integer *not null*		0
is_active	boolean *not null*		true
created_by	uuid	→ users set null	
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()

RLS **on** · 4 policies · triggers: trg_homepage_carousel_slides_updated_at

`support_requests`

Support tickets with admin reply and attachments.

id	uuid *not null*	PK	gen_random_uuid()
user_id	uuid *not null*	→ auth.users cascade	
subject	text *not null*		
message	text *not null*		
status	text *not null*		'open'::text
admin_reply	text		
created_at	timestamp with time zone *not null*		now()
updated_at	timestamp with time zone *not null*		now()
attachments	jsonb *not null*		'[]'::jsonb

RLS **on** · 4 policies · triggers: trg_touch_support_request_updated_at

`suggestions`

Feedback / suggestions sent by users (optionally anonymous) to admins or a vendor.

id	uuid *not null*	PK	gen_random_uuid()
sender_id	uuid	→ auth.users set null	
target_type	text *not null*		
target_vendor_id	uuid	→ vendors cascade	
message	text *not null*		
is_anonymous	boolean *not null*		false
status	text *not null*		'new'::text
created_at	timestamp with time zone *not null*		now()

RLS **on** · 2 policies

View

`product_rating_stats` — average rating and review count per product (reviews joined to order_items). Defined with SECURITY DEFINER semantics (flagged by the advisor; see section 5).

4. Enumerations

Type	Values
`delivery_method`	`pickup` · `delivery`
`dispute_raised_by`	`buyer` · `vendor`
`dispute_status`	`open` · `resolved`
`listing_type`	`rent` · `sale` · `both`
`order_status`	`pending` · `confirmed` · `handed_over` · `returned` · `completed` · `cancelled` · `disputed` · `payment_unresolved`
`order_type`	`rental` · `sale`
`payment_method`	`card` · `transfer` · `cash`
`payment_status`	`pending` · `paid` · `refunded`
`payment_type`	`sale_price` · `rental_fee` · `deposit` · `delivery_deposit_20`
`product_condition`	`new` · `used-good` · `used-fair`
`product_status`	`active` · `paused` · `removed`
`user_role`	`buyer` · `vendor` · `admin`
`vendor_verification_status`	`pending` · `verified` · `rejected`

Notes: `payment_type.delivery_deposit_20` is the pay-after deposit (despite the name, the percentage is stored in `orders.payment_deposit_percent`, default 20). `order_type` is sale or rental; services are orders created from an accepted service offer.

5. Security model

5.1 Row Level Security

RLS is enabled on all 71 tables, with 149 policies. Typical patterns: owners read/write their own rows (`auth.uid() = user_id`); vendors manage rows through `owns_vendor(vendor_id)`; admins through `is_admin()` / `is_super_admin()`; public catalogue tables allow anonymous `select` of active, verified-vendor data. Twelve tables intentionally have **RLS on and no policy**: they are reachable only through SECURITY DEFINER functions or the service role (e.g. `admin_login_otps`, `password_reset_requests`, `chat_archive`, `chat_usage`, `listing_payment_intents`, `points_skips`, `app_presence`, reminder log tables).

Table	Policies	Table	Policies	Table	Policies
`account_appeals`	2	`admin_actions`	2	`admin_login_otps`	0
`ai_conversations`	4	`ai_messages`	3	`app_presence`	0
`auth_activity_logs`	1	`blocked_users`	3	`bookings`	1
`cart_items`	1	`categories`	5	`chat_archive`	0
`chat_list_conversations`	1	`chat_lists`	1	`chat_usage`	0
`commission_charges`	1	`commission_deadline_reminders`	0	`commission_settlements`	1
`conversations`	2	`disputes`	3	`equipment_requests`	3
`favorites`	3	`homepage_carousel_slides`	4	`inventory`	4
`listing_billing_reminders`	0	`listing_drafts`	4	`listing_payment_intents`	0
`message_pins`	3	`messages`	3	`monthly_listing_submissions`	3
`notifications`	4	`order_items`	2	`order_notes`	1
`orders`	3	`package_comment_likes`	3	`package_comments`	4
`package_engagement_settings`	4	`package_favorites`	3	`package_items`	4
`password_reset_requests`	0	`payments`	1	`points_ledger`	1
`points_rules`	1	`points_skips`	0	`points_weekly`	1
`product_comment_likes`	3	`product_comments`	4	`product_engagement_settings`	4
`product_variants`	4	`products`	4	`push_subscriptions`	4
`referrals`	1	`request_offers`	3	`reviews`	3
`search_synonyms`	1	`sectors`	1	`service_offers`	1
`suggestions`	2	`support_requests`	4	`tts_usage`	0
`user_presence`	0	`users`	2	`vendor_follow_preferences`	4
`vendor_follows`	3	`vendor_monthly_charges`	1	`vendor_packages`	4
`vendor_profile_view_events`	1	`vendor_recycle_bin`	0	`vendor_week_winners`	1
`vendors`	3	`voice_calls`	1		

5.2 SECURITY DEFINER functions

265 of 311 functions run with the owner's rights and bypass RLS. Each must (and, by convention, does) verify the caller with `auth.uid()`, `is_admin()` or `owns_vendor()`. All use `set search_path = public` except where noted below. When adding a function, copy an existing one of the same kind (see section 8) and keep that pattern.

5.3 Supabase security advisor findings (run 7 Oct 2026)

Finding	Count	Status / recommendation
Function search_path mutable	4	**Fixed** — `validate_vendor_phone_numbers`, `set_homepage_carousel_slides_updated_at`, `points_week_start`, `generate_referral_code` now pin `search_path = public`.
SECURITY DEFINER functions executable by `anon`	223	Most check `auth.uid()` and simply fail for anonymous callers, but the surface is larger than needed. **Recommended hardening (not applied):** revoke EXECUTE from `PUBLIC, anon` on every function that requires a login, and from `authenticated` on the ~43 internal/cron-only functions and the 62 trigger functions that nothing in the client calls. Keep EXECUTE on functions used inside RLS policies and on the genuinely public ones (search, catalogue reads, view counters). A scripted audit method is in section 11.
SECURITY DEFINER functions executable by `authenticated`	241	Same hardening as above.
SECURITY DEFINER view	1	`product_rating_stats`: switch to `security_invoker = true` after confirming guests can still read ratings (the table `reviews` has public-read policy).
RLS enabled, no policy	12	Intentional (see 5.1). Add a comment or an explicit deny policy if you want the advisor quiet.
Extension in public schema	1	`pg_trgm` in `public`. Moving it requires re-creating the trigram indexes and updating functions that call `similarity()`; low risk, low priority.

Leaked-password protection disabled	1	Turn on in Auth settings.

5.4 Privacy rules enforced in the database

- Vendors cannot see a customer's name until they accept the order (`get_order_buyer_contact`, `get_orders_buyer_names`); phone numbers are admin-only.
- Chats are open only while an order is active (and 24 h after completion); messages cannot be edited or deleted freely (evidence). Admins have read-only access via `admin_list_conversations`.
- Vendors cannot buy their own products (trigger on cart + order functions). Admin accounts cannot place orders.
- Soft-deleted vendors keep their data in `vendor_recycle_bin` until an admin permanently deletes them.

6. Business logic

6.1 Order lifecycle

pending	→	confirmed	→	handed_over	→	completed
buyer placed order		vendor accepted		delivery / pickup done		buyer confirmed or 24 h elapsed

Side branches: `cancelled` (buyer within a window, vendor decline, admin), `disputed` (raise_dispute), `payment_unresolved` (vendor disputes a reported payment; escalates to admin after a timer), `returned` (rentals). Transitions go through `transition_order_status` and the dedicated functions; direct updates are blocked by RLS.

6.2 Payments (direct transfer model)

1. Order is created by `create_sale_order / create_cart_order / create_rental_order / create_package_order`. Totals, delivery fee (`compute_delivery_fee`) and payment terms (`normalize_order_payment_terms`) are computed in SQL, never trusted from the client.
2. Buyer pays the vendor outside the app and taps **I've paid** → `buyer_report_manual_payment` inserts a `payments` row with `status = pending, provider_reference = buyer_report`.
3. Vendor confirms (`vendor_confirm_reported_payment`) or disputes it (`vendor_dispute_reported_payment`). Admins can confirm/reject (`admin_confirm_reported_payment`, `admin_reject_reported_payment`). If the vendor never answers, `escalate_unresolved_payment_orders` (cron, every 5 min) marks it unresolved and escalates; `vendor_confirm_unreported_payment` lets a vendor record cash/transfer received without a buyer report once escalated.
4. **Pay-after-delivery orders** (`orders.payment_timing = pay_after`): two payments — a deposit (`delivery_deposit_20`, percentage in `payment_deposit_percent`) before the vendor starts, and the balance (`sale_price`) after delivery. The order page tracker shows: placed → confirmed → N% paid → out for delivery → pay remaining → delivered.

6.3 Completion: buyer confirmation + 24-hour auto-complete

When the **final** payment is confirmed (vendor, vendor-unreported, or admin function) the order is **not** completed immediately any more. Instead `orders.awaiting_confirmation_since = now()` and the buyer gets a notification. The buyer can:

- press **I've received this** → `buyer_confirm_received(order_id)` sets `status = completed`; or
- report a problem → `raise_dispute` (order becomes disputed; auto-completion stops).

If neither happens within 24 hours, cron job `auto-complete-orders` (every 10 min) runs `auto_complete_eligible_orders()`, which completes the order (`auto_completed_at` set) and notifies the buyer. Scope: sale and package orders on the direct-payment model; **excluded**: rentals, service-offer orders and legacy card orders whose `payout_status` is not `not_applicable` (those still release payout through `/api/order-payout-action`). Completion fires the usual triggers: commission recording (`record_order_completion_commission`), points (`trg_points_order`), review prompt.

6.4 Commission & vendor billing

- Vendors are on a **commission** model (`vendors.billing_model`) or a **monthly** listing-fee model (`products.billing_*`, `monthly_listing_submissions` approved by admins).
- Completed orders create `commission_charges`; a 7-day window applies. `process_commission_deadline_reminders` (cron 07:00 UTC) reminds at 5/3/2/1 days and on the due day; `enforce_commission_deadlines` pauses the storefront (`vendors.commission_paused`) when overdue. Settlement is paid through Paystack (`commission_settlements`).

6.5 Services & negotiation

Service listings (`products.is_service`) have no fixed price. A buyer proposes a price (`create_service_offer`); vendor and buyer counter up to 7 rounds (`respond_service_offer`); on acceptance an order is created, payment is reported/confirmed (`report_service_offer_payment`, `confirm_service_offer_payment`). Deadlines are enforced by `process_service_offer_deadlines` (cron, every 5 min).

6.6 Notifications pipeline

1. A trigger or function calls `create_notification(user, type, title, body, target_view, params, actor)` → row in `notifications` (Realtime delivers it to open apps).
2. Trigger `notify_email_on_notification` posts the row id to `/api/notify` (pg_net). The endpoint sends email (Resend) and web push (VAPID) to every `push_subscriptions` row.
3. **In-app suppression:** the app sends a heartbeat (`touch_presence`, ~20 s). `/api/notify` skips the system push if the user was active in the last 45 s (calls excepted), so open apps show notifications in-app only.
4. **Reminders:** pg_cron job `bg-reminders` (every minute) runs `send_background_reminders()`, which POSTs a batch of per-user reminder payloads (unread messages, unfinished orders) to `/api/notify` (`reminders` array, push only, one per push service per user). `remind_unattended_orders` (every 3 min) separately creates order reminder notifications.
5. Calls: `start_voice_call` creates `voice_calls` + an `incoming_call` notification; `ring_callee_again` re-pushes every 5 s while ringing (up to 9 times, 45 s timeout).

6.7 Messaging & calls

A conversation exists per (buyer, vendor). `buyer_can_message_vendor` allows chat only around an active order; `conversation_is_open` closes it 24 h after completion; `chat_purge_candidates` + archive move old chats to `chat_archive` (30-day private retention). Calls are WebRTC; signalling goes over Supabase Realtime broadcast; ICE/TURN servers come from `/api/turn` (rewritten to `api/delete-account.js`).

6.8 Points & leaderboard

`points_rules` define actions and caps. Triggers (`trg_points_*`) call `award_points`, which is idempotent through `points_ledger.event_key`, applies daily/weekly caps and logs skips. `points_weekly` feeds the leaderboard; trigger `trg_points_rank_notify` → `notify_rank_upsert` sends rank-change notifications; `finalize_points_weeks` (Sundays 23:05) writes `vendor_week_winners`; `points_reconcile_listings` (every 10 min) keeps listing points consistent.

6.9 Search & categories

`search_products_full_v2` and `search_product_ids` rank by name relevance using `pg_trgm` (`word_similarity`) plus `search_synonyms`. Trigger `trg_products_default_category` assigns uncategorised products to a default sub-category named after the vendor's sector (`ensure_default_category`), so every product appears in a category filter.

7. Scheduled jobs (pg_cron)

Job	Schedule (UTC)	Runs	Purpose
`levromart-service-offer-deadlines`	`*/5 * * * *`	`select public.process_service_offer_deadlines();`	Expire / remind stale service negotiations
`levromart-payment-unresolved-escalation`	`*/5 * * * *`	`select public.escalate_unresolved_payment_orders();`	Escalate unconfirmed buyer-reported payments to admins
`commission-deadline-reminders`	`0 7 * * *`	`select public.process_commission_deadline_reminders();`	Commission due-date reminders and enforcement
`remind-unattended-orders`	`*/3 * * * *`	`select public.remind_unattended_orders();`	Nudge vendors/buyers about orders that need action
`levromart-finalize-points-week`	`5 23 * * 0`	`select public.finalize_points_weeks()`	Close the weekly points race, pick winners
`levromart-points-reconcile`	`*/10 * * * *`	`select public.points_reconcile_listings()`	Reconcile listing points with live listings
`bg-reminders`	`* * * * *`	`select public.send_background_reminders()`	Push reminders for unread messages / unfinished orders to users who are NOT in the app
`auto-complete-orders`	`*/10 * * * *`	`select public.auto_complete_eligible_orders()`	Complete orders 24 h after final payment if the buyer did nothing and no dispute is open

Also scheduled **outside** the database: Vercel cron `/api/daily-cron` at 03:00 UTC (listing-billing reminders, old-notification cleanup); GitHub Actions `db-backup.yml` every Sunday 02:17 UTC (encrypted pg_dump, 30-day artifact) and `order-reminders.yml` (optional 30-minute trigger for `/api/order-reminders`).

> **Cost note:** `bg-reminders` makes one Vercel call per minute only when someone has something pending (~1,440/day worst case).

8. Functions, triggers & policies catalogue

311 functions in `public`: 31 are extension (pg_trgm) internals, 69 are trigger functions, the rest are RPCs and helpers. Grouped below; every name is a callable SQL object (overloads listed once).

Orders & checkout (27)

`add_order_note` `add_to_cart` `admin_cancel_confirmed_overdue_order` `auto_complete_eligible_orders` `buyer_confirm_received` `buyer_respond_quantity_proposal` `compute_delivery_fee` `compute_product_delivery_fee` `create_cart_order` `create_package_order` `create_rental_booking` `create_rental_order` `create_sale_order` `edit_buyer_order` `edit_pending_order` `get_order_buyer_contact` `get_order_vendor_payment_details` `get_orders_buyer_names` `get_reviewable_order_for_product` `remind_unattended_orders` `send_order_reminder` `set_order_buyer_location` `set_order_buyer_note` `set_order_item_variant` `transition_order_status` `vendor_decline_order` `vendor_suggest_order_quantity`

Payments, disputes & service offers (25)

`accept_request_offer` `admin_confirm_reported_payment` `admin_reject_reported_payment` `buyer_report_manual_payment` `confirm_service_offer_payment` `create_commission_payment_intent` `create_payment_intent` `create_service_offer` `dispute_service_offer_payment` `escalate_unresolved_payment_orders` `get_service_price_hint` `get_vendor_payment_details` `process_payment_unresolved_escalations` `process_service_offer_deadlines` `raise_dispute` `record_offline_payment` `report_service_offer_payment` `resolve_dispute` `respond_service_offer` `submit_monthly_listing_payment` `vendor_confirm_reported_payment` `vendor_confirm_unreported_payment` `vendor_dispute_reported_payment` `vendor_dispute_unreported_payment` `vendor_respond_to_dispute`

Commission & billing (12)

`admin_approve_monthly_listing` `admin_deactivate_monthly_listing` `admin_reject_monthly_listing` `compute_monthly_billing_amount` `enforce_commission_deadlines` `get_vendor_commission_breakdown` `get_vendor_commission_deadline` `get_vendor_commission_owed` `process_commission_deadline_reminders` `process_listing_billing_reminders` `process_vendor_commission_hour_reminders` `set_vendor_billing_model`

Vendors, users, admin & security (72)

`accept_legal_terms` `admin_adjust_points` `admin_approve_manual_password_reset` `admin_approve_vendor` `admin_award_social_points` `admin_delete_inventory_product` `admin_delete_vendor` `admin_deny_appeal` `admin_find_conversation` `admin_force_unlock_vendor` `admin_get_conversation_participants` `admin_get_inventory` `admin_get_manual_password_reset_requests` `admin_get_pending_appeals` `admin_list_conversations` `admin_list_locked_vendors` `admin_list_transactions` `admin_list_vendor_payouts` `admin_permanently_delete_vendor` `admin_recent_points_adjustments` `admin_recent_points_skips` `admin_reject_manual_password_reset` `admin_reject_vendor` `admin_restore_vendor` `admin_search_points_users` `admin_set_user_role` `admin_set_vendor_status` `admin_set_vendor_storefront_active` `admin_suspend_user` `admin_toggle_vendor_storefront_active` `admin_trash_vendor` `admin_unlink_all_identities` `admin_unsuspend_user` `am_i_blocked_with` `clear_presence` `complete_manual_password_reset` `current_role` `delete_my_account` `get_legal_terms_consent` `get_manual_password_reset_status` `get_my_account_appeal` `get_platform_suggestions` `get_vendor_follow_state` `get_vendor_follow_states` `get_vendor_follower_counts` `get_vendor_follower_network` `get_vendor_followers` `get_vendor_followers_v2` `get_vendor_product_regions` `get_vendor_spotlight` `get_vendor_stats` `get_vendor_suggestions` `increment_vendor_profile_view` `is_admin` `is_blocked_between` `is_vendor_follower` `is_vendor_owner` `mark_suggestion_read` `nearby_vendors` `owns_vendor` `record_auth_activity` `request_manual_password_reset` `resolve_account_role` `set_vendor_follow_preferences` `submit_account_appeal` `submit_suggestion` `submit_vendor_application` `sync_current_account_role` `toggle_vendor_follow` `touch_last_seen` `touch_presence` `vendor_set_storefront_active`

Catalogue, search & engagement (19)

`create_vendor_category` `create_vendor_sector` `ensure_default_category` `get_favorite_count` `get_favorite_counts` `get_or_create_custom_sector` `get_owned_product_variants` `get_package_comment_counts` `get_package_favorite_counts` `get_product_comment_counts` `get_product_reviews` `increment_product_view` `notify_product_variety_batch` `points_award_review` `save_product_variants` `search_ai_conversations` `search_product_ids` `search_products_full` `search_products_full_v2`

Messaging & calls (27)

`buyer_can_message_vendor` `can_call_in_conversation` `chat_purge_candidates` `conversation_is_open` `create_chat_list` `delete_chat_list` `delete_message` `edit_message` `get_call_party` `get_conversation_other_party` `get_conversation_presence` `get_or_create_conversation` `get_shared_ai_conversation` `increment_chat_usage` `is_conversation_participant` `list_my_chat_lists` `list_my_conversations` `mark_conversation_opened` `ring_callee_again` `set_ai_conversation_shared` `start_voice_call` `toggle_ai_message_reaction` `toggle_conversation_in_list` `toggle_conversation_pin` `toggle_conversation_unread` `toggle_message_reaction` `update_voice_call`

Notifications & reminders (6)

`create_notification` `notify_admins` `notify_points` `notify_rank_upsert` `notify_vendor_application_submitted` `send_background_reminders`

Points, leaderboard & referrals (18)

`apply_referral_code` `apply_referral_internal` `award_points` `finalize_points_weeks` `generate_referral_code` `get_my_points_activity` `get_my_points_summary` `get_past_week_winners` `get_points_leaderboard` `points_award_order` `points_for_new_listing` `points_log_skip` `points_reconcile_listings` `points_revoke_listing` `points_week_start` `record_social_click` `revoke_points` `validate_referral_code`

Levi (AI) & usage (1)

`increment_tts_usage`

Other helpers (1)

`rls_auto_enable`

Trigger functions (69)

`enforce_buyer_phone_on_order` `enforce_message_pin_limit` `enforce_vendor_onboarding_requirements` `guard_unverified_order_completion` `handle_new_levromart_user` `handle_new_user` `levro_remove_monthly_submission_on_listing_removal` `levromart_stamp_order_status` `listing_drafts_touch_updated_at` `log_auth_session_start` `mark_order_payment_unresolved_from_claim` `normalize_commission_for_billing` `normalize_order_payment_terms` `notify_admins_on_action` `notify_email_on_notification` `notify_new_dispute` `notify_new_message` `notify_new_offer` `notify_new_order` `notify_new_request` `notify_new_review` `notify_new_user_signup` `notify_new_vendor_application` `notify_offer_accepted` `notify_order_status_changed` `notify_payment_paid` `notify_product_comment_event` `notify_product_comment_like_event` `notify_product_favorite_event` `notify_request_cancelled` `notify_variant_followers_batch_aware` `notify_vendor_follow_created` `notify_vendor_followers_event` `notify_vendor_low_stock` `notify_vendor_on_follow` `notify_vendor_payout_setup` `on_user_profile_updated` `on_vendor_profile_updated` `prevent_self_role_change` `prevent_unverified_order_completion` `prevent_vendor_activation_with_owed_commission` `prevent_vendor_self_cart_item` `prevent_vendor_self_order` `products_default_category` `record_order_completion_commission` `require_buyer_phone_for_order` `restore_product_stock_on_cancelled_sale` `set_homepage_carousel_slides_updated_at` `set_request_buyer_snapshot` `set_review_buyer_snapshot` `set_updated_at_timestamp` `snapshot_product_comment_author` `sync_order_payment_deposit_percent_from_item` `sync_role_from_vendor_row` `sync_vendor_avatar_from_user` `sync_vendor_profile_avatar` `sync_vendor_review_stats` `touch_ai_conversation` `touch_service_offer_updated_at` `touch_support_request_updated_at` `trg_apply_signup_referral` `trg_points_follow` `trg_points_listing` `trg_points_order` `trg_points_rank_notify` `trg_points_review` `trg_points_vendor` `trg_vendor_referral_code` `validate_vendor_phone_numbers`

Row-security helper functions (used inside policies — keep EXECUTE granted)

`buyer_can_message_vendor` `is_admin` `is_blocked_between` `is_conversation_participant` `is_super_admin` `is_vendor_follower` `is_vendor_owner` `owns_vendor`

Triggers by table

Table	Trigger	When / what
`admin_actions`	`trg_notify_admins_on_action`	AFTER INSERT ON public.admin_actions FOR EACH ROW EXECUTE FUNCTION notify_admins_on_action()
`ai_messages`	`trg_touch_ai_conversation`	AFTER INSERT ON public.ai_messages FOR EACH ROW EXECUTE FUNCTION touch_ai_conversation()
`cart_items`	`trg_prevent_vendor_self_cart_item`	BEFORE INSERT OR UPDATE OF buyer_id, product_id ON public.cart_items FOR EACH ROW EXECUTE FUNCTION prevent_vendor_self_cart_item()
`disputes`	`trg_notify_new_dispute`	AFTER INSERT ON public.disputes FOR EACH ROW EXECUTE FUNCTION notify_new_dispute()
`equipment_requests`	`trg_notify_new_request`	AFTER INSERT ON public.equipment_requests FOR EACH ROW EXECUTE FUNCTION notify_new_request()
`equipment_requests`	`trg_notify_request_cancelled`	AFTER UPDATE ON public.equipment_requests FOR EACH ROW EXECUTE FUNCTION notify_request_cancelled()
`equipment_requests`	`trg_set_request_buyer_snapshot`	BEFORE INSERT ON public.equipment_requests FOR EACH ROW EXECUTE FUNCTION set_request_buyer_snapshot()
`favorites`	`favorites_notify_product_wishlist`	AFTER INSERT ON public.favorites FOR EACH ROW EXECUTE FUNCTION notify_product_favorite_event()
`homepage_carousel_slides`	`trg_homepage_carousel_slides_updated_at`	BEFORE UPDATE ON public.homepage_carousel_slides FOR EACH ROW EXECUTE FUNCTION set_homepage_carousel_slides_updated_at()
`listing_drafts`	`listing_drafts_touch_updated_at`	BEFORE UPDATE ON public.listing_drafts FOR EACH ROW EXECUTE FUNCTION listing_drafts_touch_updated_at()
`message_pins`	`trg_message_pins_limit`	BEFORE INSERT ON public.message_pins FOR EACH ROW EXECUTE FUNCTION

		enforce_message_pin_limit()
messages	trg_notify_new_message	AFTER INSERT ON public.messages FOR EACH ROW EXECUTE FUNCTION notify_new_message()
notifications	trg_notify_email	AFTER INSERT ON public.notifications FOR EACH ROW EXECUTE FUNCTION notify_email_on_notification()
order_items	trg_sync_order_payment_deposit_percent	AFTER INSERT OR UPDATE OF product_id ON public.order_items FOR EACH ROW EXECUTE FUNCTION sync_order_payment_deposit_percent_from_item()
orders	trg_guard_unverified_order_completion	BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION guard_unverified_order_completion()
orders	trg_levromart_stamp_order_status_clock	BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION levromart_stamp_order_status()
orders	trg_normalize_order_payment_terms	BEFORE INSERT ON public.orders FOR EACH ROW EXECUTE FUNCTION normalize_order_payment_terms()
orders	trg_notify_new_order	AFTER INSERT ON public.orders DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION notify_new_order()
orders	trg_notify_order_status_changed	AFTER UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION notify_order_status_changed()
orders	trg_orders_require_buyer_phone	BEFORE INSERT OR UPDATE OF buyer_id ON public.orders FOR EACH ROW EXECUTE FUNCTION enforce_buyer_phone_on_order()
orders	trg_points_order	AFTER UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION trg_points_order()
orders	trg_prevent_unverified_order_completion	BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION prevent_unverified_order_completion()
orders	trg_prevent_vendor_self_order	BEFORE INSERT OR UPDATE OF buyer_id, vendor_id ON public.orders FOR EACH ROW EXECUTE FUNCTION prevent_vendor_self_order()
orders	trg_record_order_completion_commission	AFTER UPDATE OF status ON public.orders FOR EACH ROW WHEN (((new.status = 'completed'::order_status) AND (old.status IS DISTINCT FROM 'completed'::ord
orders	trg_require_buyer_phone_for_order	BEFORE INSERT ON public.orders FOR EACH ROW EXECUTE FUNCTION require_buyer_phone_for_order()
orders	trg_restore_product_stock_on_cancelled_sale	AFTER UPDATE OF status ON public.orders FOR EACH ROW WHEN (((new.status = 'cancelled'::order_status) AND (old.status IS DISTINCT FROM 'cancelled'::ord
package_comments	package_comments_updated_at	BEFORE UPDATE ON public.package_comments FOR EACH ROW EXECUTE FUNCTION set_updated_at_timestamp()
package_engagement_settings	package_engagement_settings_updated_at	BEFORE UPDATE ON public.package_engagement_settings FOR EACH ROW EXECUTE FUNCTION set_updated_at_timestamp()
payments	trg_mark_order_payment_unresolved_from_claim	AFTER INSERT OR UPDATE OF status, provider_reference ON public.payments FOR EACH ROW EXECUTE FUNCTION mark_order_payment_unresolved_from_claim()
payments	trg_notify_payment_paid	AFTER INSERT OR UPDATE ON public.payments FOR EACH ROW EXECUTE FUNCTION notify_payment_paid()
points_weekly	trg_points_rank_notify	AFTER INSERT OR UPDATE OF points ON public.points_weekly FOR EACH ROW EXECUTE FUNCTION trg_points_rank_notify()
product_comment_likes	product_comment_likes_notify_event	AFTER INSERT ON public.product_comment_likes FOR EACH ROW EXECUTE FUNCTION notify_product_comment_like_event()
product_comments	product_comments_notify_event	AFTER INSERT ON public.product_comments FOR EACH ROW EXECUTE FUNCTION notify_product_comment_event()

product_comments	product_comments_snapshot_author	BEFORE INSERT ON public.product_comments FOR EACH ROW EXECUTE FUNCTION snapshot_product_comment_author()
product_engagement_settings	product_engagement_settings_updated_at	BEFORE UPDATE ON public.product_engagement_settings FOR EACH ROW EXECUTE FUNCTION set_updated_at_timestamp()
product_variants	trg_notify_low_stock_variants	AFTER UPDATE OF quantity_available ON public.product_variants FOR EACH ROW EXECUTE FUNCTION notify_vendor_low_stock()
product_variants	trg_product_variants_notify_followers	AFTER INSERT OR UPDATE OF name, photo_url, sale_price, discount_price ON public.product_variants FOR EACH ROW EXECUTE FUNCTION notify_variant_follower
products	trg_notify_low_stock_products	AFTER UPDATE OF quantity_available ON public.products FOR EACH ROW EXECUTE FUNCTION notify_vendor_low_stock()
products	trg_points_listing	AFTER INSERT OR UPDATE OF status, admin_deleted_at ON public.products FOR EACH ROW EXECUTE FUNCTION trg_points_listing()
products	trg_products_default_category	BEFORE INSERT OR UPDATE OF category_id ON public.products FOR EACH ROW WHEN ((new.category_id IS NULL)) EXECUTE FUNCTION products_default_category()
products	trg_products_notify_followers	AFTER INSERT OR UPDATE OF status, is_service, name, sale_price, rental_rate, rental_rate_unit, discount_price, description, photos, condition, listing
products	trg_products_remove_monthly_submission	AFTER DELETE OR UPDATE OF status ON public.products FOR EACH ROW EXECUTE FUNCTION levro_remove_monthly_submission_on_listing_removal()
request_offers	trg_notify_new_offer	AFTER INSERT ON public.request_offers FOR EACH ROW EXECUTE FUNCTION notify_new_offer()
request_offers	trg_notify_offer_accepted	AFTER UPDATE ON public.request_offers FOR EACH ROW EXECUTE FUNCTION notify_offer_accepted()
reviews	trg_notify_new_review	AFTER INSERT ON public.reviews FOR EACH ROW EXECUTE FUNCTION notify_new_review()
reviews	trg_points_review	AFTER INSERT OR DELETE OR UPDATE OF rating ON public.reviews FOR EACH ROW EXECUTE FUNCTION trg_points_review()
reviews	trg_set_review_buyer_snapshot	BEFORE INSERT ON public.reviews FOR EACH ROW EXECUTE FUNCTION set_review_buyer_snapshot()
reviews	trg_sync_vendor_review_stats	AFTER INSERT OR DELETE OR UPDATE ON public.reviews FOR EACH ROW EXECUTE FUNCTION sync_vendor_review_stats()
service_offers	trg_touch_service_offer_updated_at	BEFORE UPDATE ON public.service_offers FOR EACH ROW EXECUTE FUNCTION touch_service_offer_updated_at()
support_requests	trg_touch_support_request_updated_at	BEFORE UPDATE ON public.support_requests FOR EACH ROW EXECUTE FUNCTION touch_support_request_updated_at()
users	trg_notify_new_user_signup	AFTER INSERT ON public.users FOR EACH ROW EXECUTE FUNCTION notify_new_user_signup()
users	trg_prevent_self_role_change	BEFORE UPDATE ON public.users FOR EACH ROW EXECUTE FUNCTION prevent_self_role_change()
users	trg_sync_vendor_avatar_from_user	AFTER INSERT OR UPDATE OF avatar_url ON public.users FOR EACH ROW EXECUTE FUNCTION sync_vendor_avatar_from_user()
users	trg_sync_vendor_profile_avatar	AFTER INSERT OR UPDATE OF avatar_url ON public.users FOR EACH ROW EXECUTE FUNCTION sync_vendor_profile_avatar()
users	trg_user_profile_updated	AFTER UPDATE ON public.users FOR EACH ROW EXECUTE FUNCTION on_user_profile_updated()
vendor_follows	trg_points_follow	AFTER INSERT ON public.vendor_follows FOR EACH ROW EXECUTE FUNCTION trg_points_follow()
vendor_follows	vendor_follow_notify_vendor_trigger	AFTER INSERT ON public.vendor_follows FOR EACH ROW EXECUTE FUNCTION notify_vendor_on_follow()
vendor_packages	trg_packages_remove_monthly_submission	AFTER DELETE OR UPDATE OF status ON public.vendor_packages FOR EACH ROW EXECUTE

		FUNCTION levro_remove_monthly_submission_on_listing_removal()
vendor_packages	trg_points_listing	AFTER INSERT OR UPDATE OF status ON public.vendor_packages FOR EACH ROW EXECUTE FUNCTION trg_points_listing()
vendor_packages	trg_vendor_packages_notify_followers	AFTER INSERT OR UPDATE OF status, name, price, description, photos ON public.vendor_packages FOR EACH ROW EXECUTE FUNCTION notify_vendor_followers_eve
vendors	trg_enforce_vendor_onboarding_requirements	BEFORE INSERT OR UPDATE OF whatsapp_number, bank_name, bank_code, account_number, account_name, verification_status ON public.vendors FOR EACH ROW EXE
vendors	trg_notify_new_vendor_application	AFTER INSERT ON public.vendors FOR EACH ROW WHEN ((new.deleted_at IS NULL)) EXECUTE FUNCTION notify_new_vendor_application()
vendors	trg_notify_vendor_payout_setup	AFTER UPDATE OF paystack_subaccount_code ON public.vendors FOR EACH ROW WHEN (((old.paystack_subaccount_code IS NULL) AND (new.paystack_subaccount_cod
vendors	trg_points_vendor	AFTER INSERT OR UPDATE ON public.vendors FOR EACH ROW EXECUTE FUNCTION trg_points_vendor()
vendors	trg_prevent_vendor_activation_with_owed_commission	BEFORE UPDATE OF is_active, commission_paused ON public.vendors FOR EACH ROW EXECUTE FUNCTION prevent_vendor_activation_with_owed_commission()
vendors	trg_sync_user_role_from_vendor	AFTER INSERT OR UPDATE OF verification_status, is_active, deleted_at ON public.vendors FOR EACH ROW EXECUTE FUNCTION sync_role_from_vendor_row()
vendors	trg_validate_vendor_phone_numbers	BEFORE INSERT OR UPDATE OF phone, whatsapp_number ON public.vendors FOR EACH ROW EXECUTE FUNCTION validate_vendor_phone_numbers()
vendors	trg_vendor_profile_updated	BEFORE UPDATE ON public.vendors FOR EACH ROW EXECUTE FUNCTION on_vendor_profile_updated()
vendors	trg_vendor_referral_code	BEFORE INSERT OR UPDATE ON public.vendors FOR EACH ROW EXECUTE FUNCTION trg_vendor_referral_code()

8b. RLS policy listing

Command: r=select, a=insert, w=update, d=delete, *=all. Expressions truncated for readability — read the live definition with `select * from pg_policies`.

Table	Policy	Cmd	Roles	Using / check
account_appeals	account_appeals_insert_own	a	['public']	(user_id = auth.uid())
account_appeals	account_appeals_select_own	r	['public']	((user_id = auth.uid()) OR is_admin())
admin_actions	admin_actions_insert_admin	a	['auth']	(is_admin() AND (admin_id = auth.uid()))
admin_actions	admin_actions_select_admin	r	['auth']	is_admin()
ai_conversations	ai_conversations_delete_own	d	['public']	(user_id = (SELECT auth.uid() AS uid))
ai_conversations	ai_conversations_insert_own	a	['public']	(user_id = (SELECT auth.uid() AS uid))
ai_conversations	ai_conversations_select_own	r	['public']	(user_id = (SELECT auth.uid() AS uid))
ai_conversations	ai_conversations_update_own	w	['public']	(user_id = (SELECT auth.uid() AS uid))
ai_messages	ai_messages_delete_own	d	['public']	(EXISTS (SELECT 1 FROM ai_conversations c WHERE ((c.id = ai_messages.conversation_id) AND (c.user_id = (SELECT au
ai_messages	ai_messages_insert_own	a	['public']	(EXISTS (SELECT 1 FROM ai_conversations c WHERE ((c.id = ai_messages.conversation_id) AND (c.user_id = (SELECT au
ai_messages	ai_messages_select_own	r	['public']	(EXISTS (SELECT 1 FROM ai_conversations c WHERE ((c.id = ai_messages.conversation_id) AND (c.user_id = (SELECT au
auth_activity_logs	auth_activity_logs_admin_select	r	['auth']	is_admin()
blocked_users	blocked_users_delete_own	d	['public']	(blocker_id = auth.uid())
blocked_users	blocked_users_insert_own	a	['public']	((blocker_id = auth.uid()) AND (blocked_id <> auth.uid()))
blocked_users	blocked_users_select_own	r	['public']	(blocker_id = auth.uid())
bookings	bookings_select_vendor_or_admin	r	['public']	(EXISTS (SELECT 1 FROM products p WHERE ((p.id = bookings.product_id) AND (owns_vendor(p.vendor_id) OR (SELECT is
cart_items	cart_items_owner	*	['public']	(buyer_id = auth.uid())
categories	categories_delete_admin	d	['public']	(SELECT is_admin() AS is_admin)
categories	categories_insert_vendor_other	a	['auth']	((parent_category_id = '44444444-4444-4444-4444-444444444444'::uuid) AND (EXISTS (SELECT 1 FROM vendors WHERE (ven
categories	categories_select_all	r	['public']	true
categories	categories_update_admin	w	['public']	(SELECT is_admin() AS is_admin)
categories	categories_write_admin	a	['public']	(SELECT is_admin() AS is_admin)
chat_list_conversations	chat_list_conversations_owner	*	['public']	(EXISTS (SELECT 1 FROM chat_lists cl WHERE ((cl.id = chat_list_conversations.list_id) AND (cl.user_id = auth.uid()
chat_lists	chat_lists_owner	*	['public']	(user_id = auth.uid())
commission_charges	commission_charges_select	r	['public']	((SELECT owns_vendor(commission_charges.vendor_id) AS owns_vendor) OR (SELECT is_admin() AS is_admin))
commission_settlements	commission_settlements_select	r	['public']	((SELECT owns_vendor(commission_settlements.vendor_id) AS owns_vendor) OR (SELECT is_admin() AS is_admin))
conversations	conversations_insert_participant	a	['auth']	((NOT is_admin()) AND ((buyer_id = (SELECT auth.uid() AS uid)) OR (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id =
conversations	conversations_select_own	r	['public']	((buyer_id = (SELECT auth.uid() AS uid)) OR ((SELECT vendors.user_id FROM vendors WHERE (vendors.id = conversatio
disputes	disputes_insert_party	a	['public']	(EXISTS (SELECT 1 FROM orders o WHERE ((o.id = disputes.order_id) AND ((o.buyer_id = (SELECT auth.uid() AS uid))
disputes	disputes_select_party_or_admin	r	['public']	(EXISTS (SELECT 1 FROM orders o WHERE ((o.id = disputes.order_id) AND ((o.buyer_id = (SELECT auth.uid() AS uid))
disputes	disputes_update_admin	w	['public']	(SELECT is_admin() AS is_admin)
equipment_requests	requests_insert_own	a	['public']	(buyer_id = (SELECT auth.uid() AS uid))
equipment_requests	requests_select_open_or_own_or_admin	r	['public']	((status = 'open'::text) OR (buyer_id = (SELECT auth.uid() AS uid)) OR (SELECT is_admin() AS is_admin))

equipment_requests	requests_update_own_or_admin	w	['public']	((buyer_id = (SELECT auth.uid() AS uid)) OR (SELECT is_admin() AS is_admin))
favorites	favorites_delete_own	d	['public']	(buyer_id = (SELECT auth.uid() AS uid))
favorites	favorites_insert_own	a	['public']	(buyer_id = (SELECT auth.uid() AS uid))
favorites	favorites_select_own_or_admin	r	['public']	((buyer_id = (SELECT auth.uid() AS uid)) OR (SELECT is_admin() AS is_admin))
homepage_carousel_slides	carousel_slides_delete_admin	d	['public']	is_admin()
homepage_carousel_slides	carousel_slides_insert_admin	a	['public']	is_admin()
homepage_carousel_slides	carousel_slides_select_active_or_admin	r	['public']	((is_active = true) OR is_admin())
homepage_carousel_slides	carousel_slides_update_admin	w	['public']	is_admin()
inventory	Authenticated insert access	a	['auth']	(owner_id = auth.uid())
inventory	Owner delete access	d	['auth']	(owner_id = auth.uid())
inventory	Owner update access	w	['auth']	(owner_id = auth.uid())
inventory	Public read access	r	['public']	true
listing_drafts	listing_drafts_owner_delete	d	['public']	(EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = listing_drafts.vendor_id) AND (v.user_id = auth.uid()) AND (v.dele
listing_drafts	listing_drafts_owner_insert	a	['public']	(EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = listing_drafts.vendor_id) AND (v.user_id = auth.uid()) AND (v.dele
listing_drafts	listing_drafts_owner_select	r	['public']	(EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = listing_drafts.vendor_id) AND (v.user_id = auth.uid()) AND (v.dele
listing_drafts	listing_drafts_owner_update	w	['public']	(EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = listing_drafts.vendor_id) AND (v.user_id = auth.uid()) AND (v.dele
message_pins	message_pins_delete_own	d	['auth']	(user_id = auth.uid())
message_pins	message_pins_insert_own	a	['auth']	((user_id = auth.uid()) AND (EXISTS (SELECT 1 FROM (messages m JOIN conversations c ON ((c.id = m.conversation_
message_pins	message_pins_select_own	r	['auth']	((user_id = auth.uid()) AND (EXISTS (SELECT 1 FROM (messages m JOIN conversations c ON ((c.id = m.conversation_
messages	messages_insert_own	a	['public']	((sender_id = auth.uid()) AND (EXISTS (SELECT 1 FROM conversations c WHERE ((c.id = messages.conversation_id) AND
messages	messages_select_own	r	['public']	(EXISTS (SELECT 1 FROM conversations c WHERE ((c.id = messages.conversation_id) AND ((c.buyer_id = (SELECT auth.u
messages	messages_update_read_status	w	['public']	(EXISTS (SELECT 1 FROM conversations c WHERE ((c.id = messages.conversation_id) AND ((c.buyer_id = (SELECT auth.u
monthly_listing_submissions	monthly submissions admin update	w	['auth']	(EXISTS (SELECT 1 FROM users u WHERE ((u.id = auth.uid()) AND (u.role = 'admin'::user_role))))
monthly_listing_submissions	monthly submissions vendor insert own	a	['auth']	((user_id = auth.uid()) AND (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = monthly_listing_submissions.vendor_id)
monthly_listing_submissions	monthly submissions vendor select own or admin	r	['auth']	((user_id = auth.uid()) OR (EXISTS (SELECT 1 FROM users u WHERE ((u.id = auth.uid()) AND (u.role = 'admin'::user_r
notifications	notifications_delete_own	d	['auth']	(user_id = auth.uid())
notifications	notifications_insert_admin	a	['auth']	is_admin()
notifications	notifications_select_own	r	['public']	(user_id = (SELECT auth.uid() AS uid))
notifications	notifications_update_own	w	['public']	(user_id = (SELECT auth.uid() AS uid))
order_items	order_items_insert_via_order	a	['public']	(EXISTS (SELECT 1 FROM orders o WHERE ((o.id = order_items.order_id) AND (o.buyer_id = (SELECT auth.uid() AS uid)
order_items	order_items_select_via_order	r	['public']	(EXISTS (SELECT 1 FROM orders o WHERE ((o.id = order_items.order_id) AND ((o.buyer_id = (SELECT auth.uid() AS uid
order_notes	order_notes_select_participants	r	['auth']	(EXISTS (SELECT 1 FROM (orders o LEFT JOIN vendors v ON ((v.id = o.vendor_id))) WHERE ((o.id = order_notes.or

orders	orders_insert_buyer	a	['public']	(buyer_id = (SELECT auth.uid() AS uid))
orders	orders_select_party_or_admin	r	['public']	((buyer_id = (SELECT auth.uid() AS uid)) OR owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
orders	orders_update_party_or_admin	w	['public']	((buyer_id = (SELECT auth.uid() AS uid)) OR owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
package_comment_likes	package_comment_likes_delete	d	['public']	(user_id = auth.uid())
package_comment_likes	package_comment_likes_insert	a	['public']	(user_id = auth.uid())
package_comment_likes	package_comment_likes_select	r	['public']	true
package_comments	package_comments_delete	d	['public']	(user_id = auth.uid())
package_comments	package_comments_insert	a	['public']	((user_id = auth.uid()) AND (EXISTS (SELECT 1 FROM ((vendor_packages p JOIN vendors v ON ((v.id = p.vendor_id))
package_comments	package_comments_select	r	['public']	(EXISTS (SELECT 1 FROM ((vendor_packages p JOIN vendors v ON ((v.id = p.vendor_id))) LEFT JOIN package_eng
package_comments	package_comments_update	w	['public']	(user_id = auth.uid())
package_engagement_settings	package_engagement_delete	d	['public']	is_vendor_owner(vendor_id)
package_engagement_settings	package_engagement_insert	a	['public']	is_vendor_owner(vendor_id)
package_engagement_settings	package_engagement_select	r	['public']	true
package_engagement_settings	package_engagement_update	w	['public']	is_vendor_owner(vendor_id)
package_favorites	package favorites delete own	d	['public']	((buyer_id = auth.uid()) AND (NOT is_admin()))
package_favorites	package favorites insert own	a	['public']	((buyer_id = auth.uid()) AND (NOT is_admin()))
package_favorites	package favorites select own	r	['public']	((buyer_id = auth.uid()) OR is_admin())
package_items	package_items_delete_own_vendor	d	['public']	(EXISTS (SELECT 1 FROM vendor_packages vp WHERE ((vp.id = package_items.package_id) AND owns_vendor(vp.vendor_id))
package_items	package_items_insert_own_vendor	a	['public']	(EXISTS (SELECT 1 FROM vendor_packages vp WHERE ((vp.id = package_items.package_id) AND owns_vendor(vp.vendor_id))
package_items	package_items_select_via_package	r	['public']	(EXISTS (SELECT 1 FROM vendor_packages vp WHERE ((vp.id = package_items.package_id) AND ((vp.status = 'active'::pr
package_items	package_items_update_own_vendor	w	['public']	(EXISTS (SELECT 1 FROM vendor_packages vp WHERE ((vp.id = package_items.package_id) AND owns_vendor(vp.vendor_id))
payments	payments_select_party_or_admin	r	['public']	(EXISTS (SELECT 1 FROM orders o WHERE ((o.id = payments.order_id) AND ((o.buyer_id = (SELECT auth.uid() AS uid))
points_ledger	points_ledger_own	r	['auth']	(user_id = auth.uid())
points_rules	points_rules_read	r	['anon', 'auth']	true
points_weekly	points_weekly_read	r	['auth']	((user_id = auth.uid()) OR (EXISTS (SELECT 1 FROM vendors v WHERE (v.user_id = points_weekly.user_id))))
product_comment_likes	product_comment_likes_delete_own	d	['public']	(user_id = auth.uid())
product_comment_likes	product_comment_likes_insert_own	a	['public']	(user_id = auth.uid())
product_comment_likes	product_comment_likes_select_visible	r	['public']	true
product_comments	product_comments_delete_own	d	['public']	((user_id = auth.uid()) OR is_admin())
product_comments	product_comments_insert_own	a	['public']	((user_id = auth.uid()) AND (EXISTS (SELECT 1 FROM ((products p JOIN vendors v ON ((v.id = p.vendor_id)))
product_comments	product_comments_select_visible	r	['public']	true
product_comments	product_comments_update_own	w	['public']	((user_id = auth.uid()) OR is_admin())
product_engagement_settings	product_engagement_delete	d	['public']	is_vendor_owner(vendor_id)
product_engagement_settings	product_engagement_insert	a	['public']	is_vendor_owner(vendor_id)
product_engagement_settings	product_engagement_select	r	['public']	true
product_engagement_settings	product_engagement_update	w	['public']	is_vendor_owner(vendor_id)
product_variants	product_variants_delete_own_vendor_or_admin	d	['public']	(EXISTS (SELECT 1 FROM products p WHERE ((p.id = product_variants.product_id) AND (owns_vendor(p.vendor_id) OR is_

product_variants	product_variants_insert_own_vendor	a	['public']	(EXISTS (SELECT 1 FROM products p WHERE ((p.id = product_variants.product_id) AND (owns_vendor(p.vendor_id) OR is_
product_variants	product_variants_select_active_or_own	r	['public']	(EXISTS (SELECT 1 FROM products p WHERE ((p.id = product_variants.product_id) AND ((p.status = 'active'::product_s
product_variants	product_variants_update_own_vendor_or_admin	w	['public']	(EXISTS (SELECT 1 FROM products p WHERE ((p.id = product_variants.product_id) AND (owns_vendor(p.vendor_id) OR is_
products	products_delete_own_vendor_or_admin	d	['public']	(owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
products	products_insert_own_vendor	a	['public']	owns_vendor(vendor_id)
products	products_select_active_or_own	r	['public']	(((status = 'active'::product_status) AND (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = products.vendor_id) AND
products	products_update_own_vendor_or_admin	w	['public']	(owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
push_subscriptions	push_subs_delete_own	d	['public']	(auth.uid() = user_id)
push_subscriptions	push_subs_insert_own	a	['public']	(auth.uid() = user_id)
push_subscriptions	push_subs_select_own	r	['public']	(auth.uid() = user_id)
push_subscriptions	push_subs_update_own	w	['public']	(auth.uid() = user_id)
referrals	referrals_own	r	['auth']	((referrer_user_id = auth.uid()) OR (referred_user_id = auth.uid()))
request_offers	offers_insert_vendor_on_open_request	a	['public']	((SELECT owns_vendor(request_offers.vendor_id) AS owns_vendor) AND (EXISTS (SELECT 1 FROM equipment_requests r WH
request_offers	offers_select_party_or_admin	r	['public']	((SELECT owns_vendor(request_offers.vendor_id) AS owns_vendor) OR (EXISTS (SELECT 1 FROM equipment_requests r WHE
request_offers	offers_update_party_or_admin	w	['public']	((SELECT owns_vendor(request_offers.vendor_id) AS owns_vendor) OR (EXISTS (SELECT 1 FROM equipment_requests r WHE
reviews	reviews_insert_buyer_after_completed	a	['auth']	((buyer_id = auth.uid()) AND (EXISTS (SELECT 1 FROM orders o WHERE ((o.id = reviews.order_id) AND (o.buyer_id = au
reviews	reviews_insert_buyer_after_verified_payment	a	['public']	((buyer_id = auth.uid()) AND (EXISTS (SELECT 1 FROM orders o WHERE ((o.id = reviews.order_id) AND (o.buyer_id = au
reviews	reviews_select_all	r	['public']	true
search_synonyms	search_synonyms_read_all	r	['public']	true
sectors	sectors_select_all	r	['public']	true
service_offers	service_offers_select_participant	r	['auth']	((buyer_id = auth.uid()) OR (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = service_offers.vendor_id) AND (v.user_
suggestions	suggestions_insert_own	a	['public']	(sender_id = auth.uid())
suggestions	suggestions_select_admin	r	['public']	is_admin()
support_requests	support_requests_admin_select	r	['auth']	(SELECT is_admin() AS is_admin)
support_requests	support_requests_admin_update	w	['auth']	(SELECT is_admin() AS is_admin)
support_requests	support_requests_insert_own	a	['auth']	((SELECT auth.uid() AS uid) = user_id)
support_requests	support_requests_select_own	r	['auth']	(((SELECT auth.uid() AS uid) = user_id) OR (SELECT is_admin() AS is_admin))
users	users_select_self_or_admin	r	['public']	((id = (SELECT auth.uid() AS uid)) OR (SELECT is_admin() AS is_admin))
users	users_update_self	w	['public']	(id = (SELECT auth.uid() AS uid))
vendor_follow_preferences	vendor_follow_preferences_delete_own	d	['auth']	(follower_id = auth.uid())
vendor_follow_preferences	vendor_follow_preferences_insert_own	a	['auth']	((follower_id = auth.uid()) AND (NOT (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = vendor_follow_preferences.ven
vendor_follow_preferences	vendor_follow_preferences_select_own	r	['auth']	(follower_id = auth.uid())
vendor_follow_preferences	vendor_follow_preferences_update_own	w	['auth']	(follower_id = auth.uid())
vendor_follows	vendor_follows_delete_own	d	['auth']	(follower_id = auth.uid())

vendor_follows	vendor_follows_insert_own	a	['auth']	((follower_id = auth.uid()) AND (NOT (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = vendor_follows.vendor_id) AND
vendor_follows	vendor_follows_select_own	r	['auth']	(follower_id = auth.uid())
vendor_monthly_charges	vendor_monthly_charges_select_own	r	['public']	(((SELECT vendors.user_id FROM vendors WHERE (vendors.id = vendor_monthly_charges.vendor_id)) = (SELECT auth.uid(
vendor_packages	vendor_packages_delete_own_vendor_or_admin	d	['public']	(owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
vendor_packages	vendor_packages_insert_own_vendor	a	['public']	owns_vendor(vendor_id)
vendor_packages	vendor_packages_select_active_or_own	r	['public']	(((status = 'active'::product_status) AND (EXISTS (SELECT 1 FROM vendors v WHERE ((v.id = vendor_packages.vendor_i
vendor_packages	vendor_packages_update_own_vendor_or_admin	w	['public']	(owns_vendor(vendor_id) OR (SELECT is_admin() AS is_admin))
vendor_profile_view_events	Vendors can read their own profile view events	r	['public']	(vendor_id IN (SELECT vendors.id FROM vendors WHERE (vendors.user_id = auth.uid())))
vendor_week_winners	vendor_week_winners_read	r	['anon', 'auth']	true
vendors	vendors_insert_own	a	['public']	((user_id = (SELECT auth.uid() AS uid)) AND (NOT is_admin()))
vendors	vendors_select_public	r	['public']	(((verification_status = 'verified'::vendor_verification_status) AND is_active) OR (user_id = (SELECT auth.uid() AS uid
vendors	vendors_update_own_or_admin	w	['public']	((user_id = (SELECT auth.uid() AS uid)) OR (SELECT is_admin() AS is_admin))
voice_calls	voice_calls_select_participants	r	['auth']	((caller_id = auth.uid()) OR (callee_id = auth.uid()) OR is_admin())

9. Storage buckets & edge functions

Bucket	Public	Contents / use
`vendor-docs`	no	Vendor verification documents (CAC etc.). Private, signed URLs.
`product-photos`	yes	Legacy product images. New public images go to Cloudflare R2; this bucket still holds ~277 files (144 MB) — verify R2 coverage before deleting.
`avatars`	yes	User avatars.
`vendor-logos`	yes	Vendor logos.
`chat-attachments`	no	Chat images/files. Private, signed URLs.
`champion-photos`	no	LEFTOVER from an unrelated project (Mobihealth Campus Champions). 1 file. Candidate for removal.
`champion-cvs`	no	LEFTOVER from the same unrelated project. Empty. Candidate for removal.
`support-attachments`	no	Support-request attachments.
`homepage-carousel`	yes	Home banner images (admin-managed).
`listing-payment-receipts`	no	Receipts for monthly listing payments (10 MB limit; jpeg/png/webp/gif/pdf).
`dispute-evidence`	no	Dispute evidence images (10 MB limit).

32 storage RLS policies (schema `storage`) restrict who can read/write which bucket paths; private buckets are accessed through signed URLs created server-side or by RPC. Cloudflare R2 holds public product/avatar/logo/carousel images; the helpers are in `api/_util.js`, and the admin pages “R2 migration” / “R2 image size” exist to move and shrink older files.

Edge functions

Name	JWT	Purpose
`champions-notify`	required	Sends a “Mobihealth Campus Champions” application email through Resend. **Unrelated to Levromart** — a leftover from another project. Nothing in this repository calls it. Candidate for removal (together with the champion-* buckets).

10. Vercel API layer

Twelve serverless files in `api/` (Vercel Hobby cap). They all verify the caller's Supabase access token (or a shared secret for server-to-server calls) before using the service-role key. Rewrites in `vercel.json` expose some files under friendly paths.

File	Public path(s)	Responsibility
`_util.js`	–	env helper, body reader, Cloudflare R2 client
`auth-otp.js`	/api/auth-otp	Admin email OTP login verification
`chat.js`	/api/chat	Levi AI (Gemini tool-calling over the user's own data) and `mode: transcribe` (speech-to-text for Android voice input). Daily limit via `increment_chat_usage`
`daily-cron.js`	/api/daily-cron (Vercel cron 03:00)	Listing-billing reminders, old notification cleanup
`delete-account.js`	/api/delete-account, /api/register, /api/account-exists, /api/turn, share-preview pages	Account deletion, registration helpers, TURN credentials, link previews
`initiate-payment.js`, `verify-payment.js`, `paystack-webhook.js`	/api/…	Paystack: start, verify, webhook (marks payments paid)
`notify.js`	/api/notify	Email + web push sender; call ring-again; decline-call action; batched reminders; in-app suppression
`order-notify.js`, `order-reminders.js`, `order-payout-action.js`	/api/…	Order emails, reminder sweep, payout release / refund / buyer-cancel / vendor-decline
`vendor-payments.js`	/api/vendor-payments, /api/listing-payments	Listing activation, commission settlement, drafts, subaccounts, R2 presign/migrate

11. Audit: tidy-up performed and open decisions

Done on 7 Oct 2026

Change	Why
Added a descriptive `COMMENT` on all 71 tables (group letter + purpose)	Self-documenting database; visible in the Supabase dashboard and in pg_dump.
Pinned `search_path = public` on 4 functions	Fixes the advisor “mutable search_path” warnings.
Created `ensure_default_category()` + trigger `trg_products_default_category`; back-filled every uncategorised product	134 products now all have a category; vendor-created sectors such as “Gifts” appear in the home category rail and category filters.
New order completion flow: `buyer_confirm_received`, real `auto_complete_eligible_orders`, cron `auto-complete-orders`; updated the three payment-confirmation functions	Buyer confirms receipt; auto-complete after 24 h (section 6.3).
New `app_presence` table + `touch_presence` / `clear_presence`	Keeps system pushes out of an open app.
New `get_orders_buyer_names` (batch) replacing one RPC per order	Faster vendor orders page.
New cron `bg-reminders` + `send_background_reminders()`	Reminders for users outside the app.

Left for you to approve (all are removals or privilege changes, so they were not applied automatically)

Item	Detail	Suggested action
Duplicate trigram indexes	`vendor_packages` has two identical pairs: `idx_vendor_packages_name_trgm` = `vendor_packages_name_trgm_idx`, `idx_vendor_packages_description_trgm` = `vendor_packages_description_trgm_idx`	`drop index public.idx_vendor_packages_name_trgm, public.idx_vendor_packages_description_trgm;`
Duplicate sectors	“Food & Groceries” and “Photography” each exist twice; the extra rows (ids `a5a6c2b2-2c52-4c4c-b386-bc64a8760a61`, `d35d824f-4b7c-4890-9736-3d58c5cccd48`) have no categories or vendors	Delete those two rows.
Unused vendor-created sectors	~64 sectors with no vendors and no categories (created through the “Other” option)	Decide whether to keep them as selectable options; if not, delete unreferenced ones (only `vendors` and `categories` reference sectors, both NO ACTION, so deletion is safe).
Leftovers from another project	Edge function `champions-notify`; buckets `champion-photos` (1 file) and `champion-cvs` (empty)	Remove after confirming they are not needed.
Legacy objects	Table `inventory` (first-version blood/equipment stock) is not used by the marketplace; `search_products_full` v1 sits next to `_v2`; bucket `product-photos` (144 MB) predates R2	Drop after verifying nothing reads them.
Soft-deleted data	9 vendors with `deleted_at` and 75 notifications with `deleted_at`	Keep (restore semantics) or add a retention job.
Notification volume	`notifications` is the largest table (5.9k rows, ~4.8 MB); `vendor_follow_update` is 44% of it	Add a retention rule (e.g. delete read notifications older than 30–60 days) in `/api/daily-cron` or pg_cron.
Duplicate admin alerts	“A user updated their details” is inserted twice per profile update	Find the two triggers/functions on `users` that call `notify_admins` and keep one.
Function privileges	See section 5.3	Revoke EXECUTE where not needed (below).
Standby project	Out of date	Rebuild from an export or delete.

How the privilege audit was done (repeat it before applying)

1. List every SECURITY DEFINER function with its grants: `select p.oid::regprocedure, has_function_privilege('anon',p.oid,'execute') anon, has_function_privilege('authenticated',p.oid,'execute') auth from pg_proc p where pronamespace='public'::regnamespace and prosecdef;`
2. Collect every RPC name used by the clients: `grep -o "rpc(['\"][a-z_0-9]*" index.html api/*.js sw.js`.
3. Exclude functions referenced in RLS policies or column defaults (`pg_policies`, `pg_attrdef`) and event-trigger functions.
4. For the remainder run `revoke execute on function public.<name>(<args>) from public, anon, authenticated;` (service role and the owner keep access, so cron and /api keep working). Test the app after each batch.

12. Migration playbook (leaving Supabase)

Supabase is Postgres plus a managed bundle. The data and business logic are plain Postgres and port cleanly; the bundled services need replacements. This table is the checklist of every Supabase-specific dependency.

Supabase feature	How Levromart uses it	Replacement work
Auth (`auth.users`, JWT, `auth.uid()`)	Sign-up/sign-in, Google OAuth, sessions. **All 149 RLS policies and ~265 functions call `auth.uid()`.** `users.id` = `auth.users.id`.	Biggest item. Either keep a compatible `auth.uid()` (set from a JWT claim, e.g. `current_setting('request.jwt.claims')` as PostgREST does) or move checks into an API layer. Export `auth.users` + `auth.identities` (the weekly backup already does) and import into the new IdP, keeping the same UUIDs.
PostgREST (auto REST + `rpc()`)	The browser calls tables and ~200 RPCs directly through `@supabase/supabase-js`.	Run PostgREST yourself (same API, works with any Postgres) or put a backend in front and change the client calls. Self-hosting PostgREST is the lowest-effort path.
Realtime	Notifications, messages (postgres changes), call signalling (broadcast).	Replace with WebSockets / SSE (e.g. Ably, Pusher, Centrifugo, or the Supabase Realtime server self-hosted). Call signalling and the notification bell depend on it.
Storage	Private buckets (docs, chat attachments, receipts, evidence, support) + 4 public legacy buckets; policies in `storage.*`.	Move to S3/R2 with signed URLs; replace the storage RLS policies with server-side authorization. Public images already live on R2.
pg_cron	8 jobs (section 7).	Keep pg_cron if available on the target; otherwise recreate as scheduled workers calling the same SQL functions.
pg_net	DB → `/api/notify` webhooks.	Keep pg_net, or replace with `LISTEN/NOTIFY` or an outbox table polled by a worker (more reliable).
Vault / secrets	Webhook secret is hard-coded in 3 function bodies.	Move to a proper secret store before migrating.
Edge function	Only the unrelated `champions-notify`.	Nothing to port.
Extensions	pg_trgm (search), btree_gist (booking exclusion), pgcrypto, uuid-ossp.	Standard extensions; enable on the target.

Suggested order

1. **Freeze the contract:** tag the repo, take a fresh export (`supabase/export.sh`) and an encrypted backup; run the security hardening (section 5.3) on the old project first so the new one starts clean.
2. **Stand up the target Postgres 17** with the same extensions; restore the `public` schema (structure, then data) from the export.
3. **Decide the auth strategy** (see table) — this determines whether RLS can be kept as-is.
4. **Port services in this order:** storage → realtime → cron → webhooks → auth. Keep the Vercel functions (they only need `SUPABASE_URL`/keys swapped for the new REST endpoint and service credential).
5. **Run both in parallel** with the standby project as the rehearsal target; replay a day of traffic if possible; compare table counts and spot-check order totals.
6. **Cut over** in a quiet window: set the app read-only (maintenance banner), final incremental dump of changed tables, switch env vars and the URL/anon key in `index.html`, rotate the webhook secret, re-subscribe push if the origin changes.

> **Data to be careful with:** money tables (`orders`, `payments`, `commission_*`), `auth.users` password hashes (bcrypt, portable), push subscriptions (tied to the origin), and anything referencing storage object paths.

13. Operations runbook

Exporting the schema (always up to date)

`DB_URL="postgresql://postgres:<password>@db.<ref>.supabase.co:5432/postgres" ./supabase/export.sh` writes ordered SQL files to `supabase/export/` (full schema, types+tables, indexes/constraints/triggers/policies, functions, cron). **Before committing or sharing the output, replace the webhook secret** found in three functions with a placeholder.

Backups

- GitHub Actions **Weekly database backup** (Sundays 02:17 UTC): `pg_dump -Fc` of schema `public` plus data of `auth.users` / `auth.identities`, GPG-encrypted with `BACKUP_PASSPHRASE`, kept 30 days as a workflow artifact.
- Restore: `gpg -d backup.tar.gz.gpg | tar xz`, then `pg_restore` the `.dump` files into a fresh project.
- Also enable Supabase point-in-time recovery if the plan allows.

Making a database change

1. Write the SQL (functions: `create or replace function … security definer set search_path = public`; begin with an `auth.uid()` check).
2. Apply to live with the Supabase SQL editor or MCP; split large scripts into single statements.
3. Update the table `COMMENT` (or function comment) and the docs in `supabase/README.md`.
4. If the change affects what the client calls, ship the matching `index.html` change in the same deploy.
5. Re-run the advisor (Dashboard → Advisors) and fix anything new.

Common tasks

Task	How
Add a notification type	Insert via `create_notification` from a trigger; add its icon mapping in `api/notify.js` (`TYPE_ICON`); if push-only/in-app-only, extend the `callType` list there.
Add a points rule	Insert into `points_rules`; call `award_points(user, kind, event_key, …)` from a trigger.
Change payment deposit percentage	Per product: `products.payment_deposit_percent` (copied to the order by `sync_order_payment_deposit_percent_from_item`).
Change the auto-complete window	Edit the `interval '24 hours'` in `auto_complete_eligible_orders` (and the 24 h text in the UI / notifications).
Slow the reminder cadence	`select cron.alter_job(job_id, schedule := '*/2 * * * *')` for `bg-reminders`.
Rotate the webhook secret	Change `EMAIL_WEBHOOK_SECRET` in Vercel, then redefine `notify_email_on_notification`, `ring_callee_again` and `send_background_reminders` with the new value.
Make someone an admin	`admin_set_user_role` (super-admin only) or update `users.role` / `is_super_admin` in the SQL editor.

Monitoring

- Supabase Logs (API, Postgres, Auth, Realtime) — first stop for 4xx/5xx. Advisors tab for security/performance.
- Vercel function logs for `/api/*` (payments, notify, chat).
- Useful queries: failed notification pushes live only in Vercel logs; check `push_subscriptions` growth; `pg_stat_statements` for slow queries; `cron.job_run_details` for job failures.

Known quirks

- Many functions have overloads (e.g. `create_sale_order` has three) — the client may call an older signature; check `index.html` before changing arguments.
- Legacy card/payout fields on `orders` (`payout_status`, `auto_release_at`, `payout_*`) belong to the old Paystack-held-payment flow; new orders have `payout_status = not_applicable`.
- Two presence tables with different purposes (`user_presence` = last seen, `app_presence` = open-now heartbeat).
- The service worker caches the shell network-first; after a deploy users may need one reload.

Appendix: indexes

186 indexes (primary keys, uniques, foreign-key lookups, trigram search). Listed as table — definition.

Table	Index	Definition
account_appeals	account_appeals_pkey	account_appeals USING btree (id)
admin_actions	admin_actions_pkey	admin_actions USING btree (id)
admin_login_otps	admin_login_otps_pkey	admin_login_otps USING btree (id)
admin_login_otps	admin_login_otps_user_idx	admin_login_otps USING btree (user_id, created_at DESC)
ai_conversations	ai_conversations_pkey	ai_conversations USING btree (id)
ai_conversations	ai_conversations_share_token_key	ai_conversations USING btree (share_token)
ai_conversations	ai_conversations_user_id_idx	ai_conversations USING btree (user_id, updated_at DESC)
ai_messages	ai_messages_conversation_id_idx	ai_messages USING btree (conversation_id, created_at)
ai_messages	ai_messages_pkey	ai_messages USING btree (id)
app_presence	app_presence_pkey	app_presence USING btree (user_id)
auth_activity_logs	auth_activity_logs_occurred_at_idx	auth_activity_logs USING btree (occurred_at DESC)
auth_activity_logs	auth_activity_logs_pkey	auth_activity_logs USING btree (id)
auth_activity_logs	auth_activity_logs_session_id_key	auth_activity_logs USING btree (session_id)
auth_activity_logs	auth_activity_logs_user_id_idx	auth_activity_logs USING btree (user_id, occurred_at DESC)
blocked_users	blocked_users_pkey	blocked_users USING btree (blocker_id, blocked_id)
bookings	bookings_order_item_id_idx	bookings USING btree (order_item_id)
bookings	bookings_pkey	bookings USING btree (id)
bookings	bookings_product_range_idx	bookings USING gist (product_id, date_range)
cart_items	cart_items_buyer_idx	cart_items USING btree (buyer_id)
cart_items	cart_items_buyer_product_no_variant_unique	cart_items USING btree (buyer_id, product_id) WHERE (variant_id IS NULL)
cart_items	cart_items_buyer_product_variant_unique	cart_items USING btree (buyer_id, product_id, variant_id) WHERE (variant_id IS NOT NULL)
cart_items	cart_items_pkey	cart_items USING btree (id)
categories	categories_parent_category_id_idx	categories USING btree (parent_category_id)
categories	categories_pkey	categories USING btree (id)
categories	idx_categories_name_trgm	categories USING gin (name gin_trgm_ops)
chat_archive	chat_archive_pkey	chat_archive USING btree (id)
chat_list_conversations	chat_list_conversations_pkey	chat_list_conversations USING btree (list_id, conversation_id)
chat_lists	chat_lists_pkey	chat_lists USING btree (id)
chat_lists	chat_lists_user_name_uniq	chat_lists USING btree (user_id, lower(name))
chat_usage	chat_usage_pkey	chat_usage USING btree (user_id, day)
commission_charges	commission_charges_pkey	commission_charges USING btree (id)
commission_deadline_reminders	commission_deadline_reminders_pkey	commission_deadline_reminders USING btree (id)
commission_deadline_reminders	commission_deadline_reminders_vendor_id_reminder_code_perio_key	commission_deadline_reminders USING btree (vendor_id, reminder_code, period_key)
commission_settlements	commission_settlements_pkey	commission_settlements USING btree (id)
commission_settlements	commission_settlements_provider_reference_key	commission_settlements USING btree (provider_reference)
conversations	conversations_buyer_id_vendor_id_key	conversations USING btree (buyer_id, vendor_id)
conversations	conversations_pkey	conversations USING btree (id)
disputes	disputes_order_id_idx	disputes USING btree (order_id)
disputes	disputes_pkey	disputes USING btree (id)
equipment_requests	equipment_requests_pkey	equipment_requests USING btree (id)
favorites	favorites_buyer_id_product_id_key	favorites USING btree (buyer_id, product_id)
favorites	favorites_pkey	favorites USING btree (id)

homepage_carousel_slides	homepage_carousel_slides_active_order_idx	homepage_carousel_slides USING btree (is_active, sort_order)
homepage_carousel_slides	homepage_carousel_slides_pkey	homepage_carousel_slides USING btree (id)
inventory	inventory_created_at_idx	inventory USING btree (created_at DESC)
inventory	inventory_pkey	inventory USING btree (id)
inventory	inventory_type_idx	inventory USING btree (type)
listing_billing_reminders	listing_billing_reminders_pkey	listing_billing_reminders USING btree (kind, listing_id, reminder_code, period_key)
listing_billing_reminders	listing_billing_reminders_sent_idx	listing_billing_reminders USING btree (sent_at DESC)
listing_drafts	listing_drafts_pkey	listing_drafts USING btree (id)
listing_drafts	listing_drafts_vendor_created_idx	listing_drafts USING btree (vendor_id, created_at DESC)
listing_payment_intents	listing_payment_intents_pkey	listing_payment_intents USING btree (id)
listing_payment_intents	listing_payment_intents_reference_key	listing_payment_intents USING btree (reference)
listing_payment_intents	listing_payment_intents_user_idx	listing_payment_intents USING btree (user_id, created_at DESC)
listing_payment_intents	listing_payment_intents_vendor_idx	listing_payment_intents USING btree (vendor_id, created_at DESC)
listing_payment_intents	listing_payment_intents_vendor_pending_idx	listing_payment_intents USING btree (vendor_id, status, created_at DESC)
message_pins	message_pins_expires_at_idx	message_pins USING btree (expires_at)
message_pins	message_pins_pkey	message_pins USING btree (id)
message_pins	message_pins_user_conversation_idx	message_pins USING btree (user_id, conversation_id, pinned_at DESC)
message_pins	message_pins_user_id_message_id_key	message_pins USING btree (user_id, message_id)
messages	messages_conversation_idx	messages USING btree (conversation_id, created_at)
messages	messages_pkey	messages USING btree (id)
messages	messages_reply_to_id_idx	messages USING btree (reply_to_id)
monthly_listing_submissions	monthly_listing_submissions_listing_idx	monthly_listing_submissions USING btree (listing_id, submitted_at DESC)
monthly_listing_submissions	monthly_listing_submissions_one_pending_idx	monthly_listing_submissions USING btree (listing_id) WHERE (status = 'pending'::text)
monthly_listing_submissions	monthly_listing_submissions_pkey	monthly_listing_submissions USING btree (id)
monthly_listing_submissions	monthly_listing_submissions_status_idx	monthly_listing_submissions USING btree (status, submitted_at DESC)
notifications	notifications_pkey	notifications USING btree (id)
notifications	notifications_user_unread_idx	notifications USING btree (user_id, read, created_at DESC)
notifications	notifications_vendor_group_idx	notifications USING btree (user_id, vendor_id, notification_category, group_key, created_at DESC)
order_items	order_items_order_id_idx	order_items USING btree (order_id)
order_items	order_items_pkey	order_items USING btree (id)
order_items	order_items_product_id_idx	order_items USING btree (product_id)
order_items	order_items_variant_id_idx	order_items USING btree (variant_id)
order_notes	order_notes_order_idx	order_notes USING btree (order_id, created_at)
order_notes	order_notes_pkey	order_notes USING btree (id)
orders	idx_orders_payment_unresolved_at	orders USING btree (payment_unresolved_at) WHERE (payment_unresolved_at IS NOT NULL)
orders	idx_orders_payment_unresolved_escalation	orders USING btree (payment_unresolved_at) WHERE (status = 'payment_unresolved'::order_status)
orders	orders_buyer_id_idx	orders USING btree (buyer_id)
orders	orders_pkey	orders USING btree (id)
orders	orders_vendor_id_idx	orders USING btree (vendor_id)
package_comment_likes	package_comment_likes_comment_id_idx	package_comment_likes USING btree (comment_id)
package_comment_likes	package_comment_likes_comment_id_user_id_key	package_comment_likes USING btree (comment_id, user_id)

package_comment_likes	package_comment_likes_pkey	package_comment_likes USING btree (id)
package_comments	package_comments_package_id_created_at_idx	package_comments USING btree (package_id, created_at DESC)
package_comments	package_comments_parent_comment_id_idx	package_comments USING btree (parent_comment_id)
package_comments	package_comments_pkey	package_comments USING btree (id)
package_engagement_settings	package_engagement_settings_pkey	package_engagement_settings USING btree (package_id)
package_engagement_settings	package_engagement_settings_vendor_id_idx	package_engagement_settings USING btree (vendor_id)
package_favorites	package_favorites_buyer_created_idx	package_favorites USING btree (buyer_id, created_at DESC)
package_favorites	package_favorites_buyer_id_package_id_key	package_favorites USING btree (buyer_id, package_id)
package_favorites	package_favorites_package_idx	package_favorites USING btree (package_id)
package_favorites	package_favorites_pkey	package_favorites USING btree (id)
package_items	package_items_package_id_idx	package_items USING btree (package_id)
package_items	package_items_pkey	package_items USING btree (id)
package_items	package_items_product_id_idx	package_items USING btree (product_id)
password_reset_requests	password_reset_requests_email_created_idx	password_reset_requests USING btree (lower(email), created_at DESC)
password_reset_requests	password_reset_requests_pkey	password_reset_requests USING btree (id)
password_reset_requests	password_reset_requests_status_created_idx	password_reset_requests USING btree (status, created_at DESC)
payments	payments_one_paid_fee_per_order_idx	payments USING btree (order_id, payment_type) WHERE ((status = 'paid'::payment_status) AND (payment_type = ANY (ARRAY['sale_price'
payments	payments_order_id_idx	payments USING btree (order_id)
payments	payments_pkey	payments USING btree (id)
points_ledger	points_ledger_kind_idx	points_ledger USING btree (user_id, kind, week_start)
points_ledger	points_ledger_pkey	points_ledger USING btree (id)
points_ledger	points_ledger_user_id_event_key_key	points_ledger USING btree (user_id, event_key)
points_ledger	points_ledger_user_time_idx	points_ledger USING btree (user_id, created_at DESC)
points_ledger	points_ledger_week_idx	points_ledger USING btree (week_start)
points_rules	points_rules_pkey	points_rules USING btree (kind)
points_skips	points_skips_kind_ref_key	points_skips USING btree (kind, ref)
points_skips	points_skips_pkey	points_skips USING btree (id)
points_weekly	points_weekly_pkey	points_weekly USING btree (user_id, week_start)
points_weekly	points_weekly_week_idx	points_weekly USING btree (week_start, points DESC)
product_comment_likes	product_comment_likes_comment_id_user_id_key	product_comment_likes USING btree (comment_id, user_id)
product_comment_likes	product_comment_likes_comment_idx	product_comment_likes USING btree (comment_id)
product_comment_likes	product_comment_likes_pkey	product_comment_likes USING btree (id)
product_comments	product_comments_parent_idx	product_comments USING btree (parent_comment_id)
product_comments	product_comments_pkey	product_comments USING btree (id)
product_comments	product_comments_product_created_idx	product_comments USING btree (product_id, created_at DESC)
product_engagement_settings	product_engagement_settings_pkey	product_engagement_settings USING btree (product_id)
product_engagement_settings	product_engagement_settings_vendor_id_idx	product_engagement_settings USING btree (vendor_id)
product_variants	product_variants_pkey	product_variants USING btree (id)
product_variants	product_variants_product_id_idx	product_variants USING btree (product_id, sort_order, created_at)
products	idx_products_description_trgm	products USING gin (description gin_trgm_ops)

products	idx_products_name_trgm	products USING gin (name gin_trgm_ops)
products	idx_products_vendor_created_at	products USING btree (vendor_id, created_at DESC)
products	products_category_id_idx	products USING btree (category_id)
products	products_pkey	products USING btree (id)
products	products_vendor_id_idx	products USING btree (vendor_id)
products	products_vendor_status_billing_idx	products USING btree (vendor_id, status, billing_model, billing_paid_until)
push_subscriptions	push_subscriptions_endpoint_key	push_subscriptions USING btree (endpoint)
push_subscriptions	push_subscriptions_pkey	push_subscriptions USING btree (id)
referrals	referrals_pkey	referrals USING btree (referred_user_id)
referrals	referrals_referrer_idx	referrals USING btree (referrer_user_id, created_at DESC)
request_offers	request_offers_pkey	request_offers USING btree (id)
request_offers	request_offers_request_id_vendor_id_key	request_offers USING btree (request_id, vendor_id)
reviews	reviews_buyer_id_idx	reviews USING btree (buyer_id)
reviews	reviews_order_id_key	reviews USING btree (order_id)
reviews	reviews_pkey	reviews USING btree (id)
reviews	reviews_vendor_id_idx	reviews USING btree (vendor_id)
search_synonyms	search_synonyms_pkey	search_synonyms USING btree (term, synonym)
sectors	sectors_pkey	sectors USING btree (id)
sectors	sectors_slug_key	sectors USING btree (slug)
service_offers	service_offers_buyer_idx	service_offers USING btree (buyer_id, updated_at DESC)
service_offers	service_offers_expiry_idx	service_offers USING btree (status, expires_at)
service_offers	service_offers_listing_status_idx	service_offers USING btree (listing_id, status, updated_at DESC)
service_offers	service_offers_order_unique	service_offers USING btree (order_id)
service_offers	service_offers_pkey	service_offers USING btree (id)
service_offers	service_offers_vendor_idx	service_offers USING btree (vendor_id, updated_at DESC)
suggestions	suggestions_pkey	suggestions USING btree (id)
support_requests	support_requests_pkey	support_requests USING btree (id)
support_requests	support_requests_status_created_idx	support_requests USING btree (status, created_at DESC)
support_requests	support_requests_user_created_idx	support_requests USING btree (user_id, created_at DESC)
tts_usage	tts_usage_pkey	tts_usage USING btree (user_id, day)
user_presence	user_presence_pkey	user_presence USING btree (user_id)
users	users_email_key	users USING btree (email)
users	users_phone_key	users USING btree (phone)
users	users_pkey	users USING btree (id)
vendor_follow_preferences	vendor_follow_preferences_follower_id_vendor_id_key	vendor_follow_preferences USING btree (follower_id, vendor_id)
vendor_follow_preferences	vendor_follow_preferences_pkey	vendor_follow_preferences USING btree (id)
vendor_follow_preferences	vendor_follow_preferences_vendor_idx	vendor_follow_preferences USING btree (vendor_id, follower_id)
vendor_follows	vendor_follows_follower_id_vendor_id_key	vendor_follows USING btree (follower_id, vendor_id)
vendor_follows	vendor_follows_follower_idx	vendor_follows USING btree (follower_id, created_at DESC)
vendor_follows	vendor_follows_pkey	vendor_follows USING btree (id)
vendor_follows	vendor_follows_vendor_idx	vendor_follows USING btree (vendor_id, created_at DESC)
vendor_monthly_charges	vendor_monthly_charges_pkey	vendor_monthly_charges USING btree (id)
vendor_monthly_charges	vendor_monthly_charges_vendor_idx	vendor_monthly_charges USING btree (vendor_id, created_at DESC)
vendor_packages	idx_vendor_packages_description_trgm	vendor_packages USING gin (description gin_trgm_ops)

vendor_packages	idx_vendor_packages_name_trgm	vendor_packages USING gin (name gin_trgm_ops)
vendor_packages	vendor_packages_description_trgm_idx	vendor_packages USING gin (description gin_trgm_ops)
vendor_packages	vendor_packages_name_trgm_idx	vendor_packages USING gin (name gin_trgm_ops)
vendor_packages	vendor_packages_pkey	vendor_packages USING btree (id)
vendor_packages	vendor_packages_status_created_idx	vendor_packages USING btree (status, created_at DESC)
vendor_packages	vendor_packages_vendor_status_billing_idx	vendor_packages USING btree (vendor_id, status, billing_model, billing_paid_until)
vendor_packages	vendor_packages_vendor_status_created_idx	vendor_packages USING btree (vendor_id, status, created_at DESC)
vendor_profile_view_events	vendor_profile_view_events_pkey	vendor_profile_view_events USING btree (id)
vendor_profile_view_events	vendor_profile_view_events_vendor_time_idx	vendor_profile_view_events USING btree (vendor_id, viewed_at DESC)
vendor_recycle_bin	vendor_recycle_bin_deleted_at_idx	vendor_recycle_bin USING btree (deleted_at DESC)
vendor_recycle_bin	vendor_recycle_bin_pkey	vendor_recycle_bin USING btree (vendor_id)
vendor_week_winners	vendor_week_winners_pkey	vendor_week_winners USING btree (week_start)
vendors	idx_vendors_business_name_trgm	vendors USING gin (business_name gin_trgm_ops)
vendors	idx_vendors_sector_id	vendors USING btree (sector_id)
vendors	vendors_pkey	vendors USING btree (id)
vendors	vendors_referral_code_key	vendors USING btree (referral_code)
vendors	vendors_user_id_key	vendors USING btree (user_id)
voice_calls	voice_calls_conv_idx	voice_calls USING btree (conversation_id, created_at DESC)
voice_calls	voice_calls_pkey	voice_calls USING btree (id)